

TRƯỜNG ĐẠI CÔNG NGHỆ KỸ THUẬT TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

-----□□□□-----



HCMUTE

BÁO CÁO CUỐI KỲ MÔN BIG DATA STORAGE AND RETRIEAL

Giảng viên hướng dẫn:

Quách Đình Hoàng

Sinh viên thực hiện:

Đặng Cửu Dương 22110124

Phan Văn Quân – 22110214

Trịnh Hữu Thọ - 22110238

TP. Hồ Chí Minh, tháng 1 năm 2026

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

-----□□□□-----



BÁO CÁO CUỐI KỲ MÔN BIG DATA STORAGE AND RETRIEAL

Giảng viên hướng dẫn:

Quách Đình Hoàng

Sinh viên thực hiện:

Đặng Cửu Dương – 22110124

Phan Văn Quân – 22110214

Trịnh Hữu Thọ - 22110238

TP. Hồ Chí Minh, tháng 1 năm 2026

LỜI CẢM ƠN

Trước hết, để hoàn thành được bài báo cáo này nhóm sinh viên xin chân thành cảm ơn Khoa Công nghệ Thông tin, Trường Đại học Sư Phạm Kỹ Thuật TP. Hồ Chí Minh, đã tạo điều kiện học tập và nghiên cứu, đồng thời cung cấp môi trường lý tưởng để nhóm xây dựng kiến thức và bộc lộ kỹ năng trong lĩnh vực Công nghệ Thông tin.

Đặc biệt, xin được bày tỏ lòng biết ơn sâu sắc đến thầy Quách Đình Hoàng, giảng viên hướng dẫn môn “Big Data”, người đã luôn tận tình hướng dẫn, theo sát quá trình và truyền cảm hứng cho nhóm sinh viên thực hiện báo cáo.

Sự tận tâm và nhiệt huyết của thầy không chỉ giúp nhóm nắm bắt được các kiến thức cốt lõi, xây dựng nền tảng vững chắc cho đến cách ứng dụng từng kiến thức vào thực tế sao cho hợp lý. Những góp ý của thầy trong suốt quá trình thực hiện đề tài đã giúp cho nhóm hiểu sâu sắc hơn về chủ đề mà nhóm tìm hiểu.

Một lần nữa, nhóm xin chân thành cảm ơn thầy và Khoa Công nghệ Thông tin. Hy vọng trong những chặng đường học tập và nghiên cứu sắp tới, nhóm vẫn sẽ được sự đồng hành và hướng dẫn từ quý thầy cô cũng như đội ngũ cán bộ giảng viên của khoa Công nghệ Thông tin.

Trân trọng cảm ơn!

Nhóm sinh viên thực hiện đề tài

Phan Văn Quân

Đặng Cửu Dương

Trịnh Hữu Thọ

MỤC LỤC

LỜI CẢM ƠN.....	i
MỤC LỤC	ii
DANH MỤC HÌNH	v
CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ CƠ SỞ DỮ LIỆU	1
1.1. Động lực ra đời.....	1
1.2. Các loại mô hình cơ sở dữ liệu.....	2
1.3. NoSQL được phân thành 4 loại chính sau.....	2
1.4. Sự khác biệt giữa cơ sở dữ liệu quan hệ (SQL) và phi quan hệ (NoSQL).....	3
CHƯƠNG 2. VỀ CẤU TRÚC LƯU TRỮ DỮ LIỆU	6
2.1. Những điểm nổi bật trong cấu trúc lưu trữ dữ liệu.....	6
2.2. Dữ liệu dạng Key values	8
2.2.1. World simplest database (Log structure)	8
2.2.2. Hash index	10
2.2.3. SSTable và LSMtree	14
2.2.3.1. Cách LSM tree hoạt động.....	14
2.2.3.2. Khi nào nên sử dụng LSM-tree	18
2.2.3.3. Ưu và nhược điểm	18
2.2.4. B-Tree.....	19
2.2.4.1. Cách B-tree hoạt động.....	19
2.2.4.2. Khi nào nên sử dụng B-tree.....	22
2.2.4.3. Ưu và nhược điểm	22
2.2.4.4. So sánh giữa LSM-tree và B-tree	23
2.3. Dữ liệu hướng cột (Column-oriented data)	24

2.3.1. Lưu trữ dữ liệu hướng cột là gì	24
2.3.2. Ưu điểm so với cơ chế lưu truyền thống	26
2.3.3. Khi nào nên sử dụng dữ liệu hướng cột	27
2.3.4. Một số kỹ thuật trong lưu trữ hướng cột	27
2.3.4.1. Nén cột (Column compression).....	27
2.3.4.2. Thứ tự sắp xếp (sort order).....	29
2.3.4.3. Cơ chế ghi (writing)	30
2.3.4.4. Khối dữ liệu và khung nhìn vật chất hóa (Data Cubes and Materialized Views)	30
CHƯƠNG 3. OLTP và OLAP	34
3.1. OLTP (Online Transaction Processing)	34
3.1.1. Tổng quan về OLTP	34
3.1.2. Đặc điểm của OLTP	34
3.1.3. Ưu điểm và nhược điểm	35
3.1.3.1. Ưu điểm:.....	35
3.1.3.2. Nhược điểm:.....	35
3.2. OLAP (Online Analytical Processing)	35
3.2.1. Tổng quan về OLAP.....	35
3.2.2. Đặc điểm của OLAP.....	36
3.2.3. Ưu điểm và nhược điểm	37
3.2.3.1. Ưu điểm:.....	37
3.2.3.2. Nhược điểm:.....	37
CHƯƠNG 4. DATA WAREHOUSE	38
4.1. Tổng quan về Data Warehouse	38

4.2. Kiến trúc của Data Warehouse	41
4.3. Chức năng của Data Warehouse.....	42
4.4. Các mô hình dữ liệu kinh điển	42
4.4.1. Star Schema	42
4.4.1.1. Tổng quan về Star Schema	42
4.4.1.2. Đặc điểm của Star Schema	44
4.4.1.3. Ưu điểm và nhược điểm	45
4.4.2. Snowflake	46
4.4.2.1. Ưu điểm và nhược điểm của mô hình Snowflake	46
4.4.3. So sánh Star Schema và Snowflake	47
4.5. Ưu điểm và nhược điểm của Data Warehouse	48
4.5.1. Ưu điểm	48
4.5.2. Nhược điểm	48
KẾT LUẬN	49
Tóm tắt nội dung	49
Explain map	51
TÀI LIỆU THAM KHẢO	52

DANH MỤC HÌNH

Hình 2.1. Mô hình cơ sở dữ liệu cơ bản	8
Hình 2.2. Lưu trữ và truy xuất trong mô hình cơ sở dữ liệu cơ bản	9
Hình 2.3. Cách Hash Table lưu và tra cứu dữ liệu	10
Hình 2.4. Quá trình chèn (insert) một bản ghi mới vào cấu trúc băm mở	12
Hình 2.5. Quá trình chèn (insert) một bản ghi mới vào cấu trúc băm đóng.....	12
Hình 2.6. Quá trình chèn (insert) một bản ghi mới vào cấu trúc băm động.....	14
Hình 2.7. Minh họa cấu trúc cây nhị phân cân bằng AVL.....	15
Hình 2.8. Cấu trúc của một cây đỏ đen (RB-tree).....	16
Hình 2.9. Minh họa quá trình memtable được lưu xuống đĩa	17
Hình 2.10. Minh họa quá trình nén các SSTable	18
Hình 2.11. Hình minh họa quá trình tìm khóa trong B-tree	19
Hình 2.12. Hình minh họa quá trình chèn khóa mới vào B-tree	20
Hình 2.13. Minh họa quá trình xóa khóa tại nút lá.....	21
Hình 2.14. Minh họa quá trình xóa khóa tại nút không phải lá.....	22
Hình 2.15. Bảng so sánh LSM-tree và B-tree	24
Hình 2.16. Minh họa cấu trúc dữ liệu truyền thống	25
Hình 2.17. Minh họa cấu trúc dữ liệu hướng cột	25
Hình 2.18. Một ví dụ khác của cấu trúc dữ liệu hướng cột.....	25
Hình 2.19. Bảng so sánh cấu trúc dữ liệu hướng hàng và hướng cột.....	27
Hình 2.20. Lưu trữ dữ liệu dạng nén cột được lập chỉ mục bằng bitmap.	28
Hình 2.21. Hai chiều của khối dữ liệu , tổng hợp dữ liệu bằng phép cộng.....	32
Hình 4.1. Bảng so sánh Star Schema và Snowflake Schema [3]	48

CHƯƠNG 1. GIỚI THIỆU CHUNG VỀ CƠ SỞ DỮ LIỆU

1.1. Động lực ra đời

Trong kỷ nguyên số, động lực tiên quyết và xuyên suốt lịch sử phát triển của các hệ thống cơ sở dữ liệu là năng lực giải quyết song hành hai yêu cầu then chốt: lưu trữ dữ liệu một cách an toàn, có tổ chức và truy xuất dữ liệu nhanh chóng, chính xác nhằm phục vụ hiệu quả các nghiệp vụ. Tuy nhiên, trước sự bùng nổ về khối lượng và độ phức tạp của dữ liệu, thị trường buộc phải phân hóa thành các mô hình chuyên biệt, tối ưu hóa triệt để cho từng loại Tải làm việc (Workload). Sự phân hóa này thể hiện rõ nét nhất qua hai cực đối lập: Hệ thống Xử lý Giao dịch Trực tuyến (OLTP), tập trung vào việc xử lý chính xác và kịp thời một lượng lớn giao dịch nhỏ lẻ, và Hệ thống Xử lý Phân tích Trực tuyến (OLAP), chuyên biệt hóa cho việc thống kê, tổng hợp và phân tích trên các kho dữ liệu khổng lồ nhằm hỗ trợ các quyết định chiến lược.

Chính sự khác biệt căn bản về bản chất của tải làm việc đã định hình nên kiến trúc lưu trữ, dẫn đến sự ra đời của hai trường phái chủ đạo: SQL/Cơ sở dữ liệu Quan hệ Truyền thống (RDB) và NoSQL. RDB, với nền tảng là ngôn ngữ SQL và cấu trúc chỉ mục B-Tree, được thiết kế nhằm tối đa hóa hiệu suất cho OLTP bằng cách đảm bảo tính toàn vẹn dữ liệu nghiêm ngặt (ACID) và duy trì hiệu suất tìm kiếm ổn định (Read-heavy). Ngược lại, NoSQL nổi lên như một giải pháp thay thế, tập trung khắc phục những hạn chế cố hữu của RDB về khả năng mở rộng (Scalability) và tốc độ ghi dữ liệu trong môi trường Big Data. Các hệ thống NoSQL, thường sử dụng cấu trúc LSM-Tree, ưu tiên chiến lược ghi tuần tự (Write-heavy) và linh hoạt về lược đồ (schema-less), cho phép chúng dễ dàng mở rộng quy mô và xử lý hiệu quả khối lượng lớn dữ liệu ghi/sửa, dù chấp nhận sự đánh đổi nhất định về độ phức tạp của thao tác đọc.

Trong thế giới của CSDL hiện tại ta có hai loại giải pháp chính đó là: SQL và NoSQL (cơ sở dữ liệu quan hệ và cơ sở dữ liệu phi quan hệ). Cả hai đều rất khác nhau về cách chúng được xây dựng, loại dữ liệu nó chúng lưu trữ và cả cách thức lưu trữ dữ liệu của chúng.

1.2. Các loại mô hình cơ sở dữ liệu

Cơ sở dữ liệu quan hệ lưu trữ những dữ liệu có cấu trúc rõ ràng ví dụ như danh bạ điện thoại thì sẽ chắc chắn sẽ chứa tên người và số điện thoại của người đó.

Còn cơ sở dữ liệu phi quan hệ lưu trữ giữ liệu không có cấu trúc, cấu trúc của nó rất động và phân tán, ví dụ như chứa một tệp tài liệu (document) về người dùng như số điện thoại, địa chỉ, hay những activity của họ trên Facebook hay các trang mua sắm trực tuyến. Dữ liệu của nó thường dưới dạng key-value dạng JSON, và những document có thể được tạo mà không cần phải xác định trước cấu trúc.

Cơ sở dữ liệu quan hệ chứa dữ liệu theo dạng dòng (rows) và cột (columns). Mỗi dòng sẽ chứa những thông tin của dữ liệu và mỗi cột sẽ chứa những đặc điểm của dữ liệu, giữa các bảng có thể có liên kết với nhau qua khóa ngoại (foreign key). Một số Cơ sở dữ liệu quan hệ phổ biến là MySQL, Oracle, MS SQL Server, SQLite, Postgresql, MariaDB và IBM DB2.

NoSQL là một DMS không quan hệ, không yêu cầu một lược đồ cố định, tránh các phép nối, và dễ dàng mở rộng. Cơ sở dữ liệu NoSQL được sử dụng cho các kho dữ liệu phân tán với nhu cầu lưu trữ dữ liệu khổng lồ. NoSQL được sử dụng cho dữ liệu lớn và các ứng dụng web thời gian thực. Ví dụ như các công ty như Twitter, Facebook, Google thu thập hàng terabyte dữ liệu người dùng mỗi ngày.

1.3. NoSQL được phân thành 4 loại chính sau

Key-Value Stores: Dữ liệu sẽ được lưu dưới dạng cặp key-value, với 'key' là thuộc tính duy nhất để liên kết với giá trị (value) của nó. Các Key-Value stores nổi tiếng bao gồm Redis hay Amazon DynamoDB.

Document Databases: Là cơ sở dữ liệu chứa dữ liệu dạng theo kiểu các tệp tài liệu (document) để thay thế cho kiểu dòng và cột trong table và nhiều document gộp lại sẽ thành một Collection. Khác với kiểu table là mỗi document sẽ chứa các dạng cấu trúc khác nhau. Ví dụ trong MongoDB thì mỗi document sẽ chứa dữ liệu kiểu JSON, tức là mỗi document là một JSON. Document DB tiêu biểu hiện tại có MongoDB và CouchDB.

Wide-Column Databases: Wide-Column hay còn gọi là Big-Table là mô hình dữ liệu để lưu trữ dữ liệu với khả năng mỗi rows chứa rất nhiều column(cột) mỗi cột lại là một cặp key-value, và ở đây số lượng cột là tùy biến (dynamic) cho mỗi rows, có nghĩa là số cột ở mỗi rows là khác nhau. Có thể thấy nó giống như kiểu key-value hai chiều với mỗi key lại chứa nhiều cặp key-value bên trong. Các Wide-Column DB tiêu biểu bao gồm Cassandra hay HBase.

Graph Databases: cơ sở dữ liệu dạng này sẽ lưu dữ liệu dạng cấu trúc dữ liệu kiểu đồ thị (graph) để biểu thị mối quan hệ giữa các dữ liệu với nhau. Dữ liệu được lưu vào các nodes (entities) và thuộc tính của node, liên kết giữa các nodes sẽ qua các lines. Graph DB phổ biến có Neo4J và InfiniteGraph.

1.4. Sự khác biệt giữa cơ sở dữ liệu quan hệ (SQL) và phi quan hệ (NoSQL)

Lưu trữ (Storage): SQL lưu trữ dữ liệu trong các bảng (tables) trong đó mỗi row đại diện cho một thực thể dữ liệu (entity), và mỗi column sẽ chứa các thuộc tính của entity; ví dụ nếu ta chứa dữ liệu của thực thể xe hơi (car entity) ta sẽ lưu trữ vào một bảng dữ liệu có nhiều column đại diện cho đặc điểm của chiếc xe đó như “màu” “nhãn hiệu” “hãng sản xuất”. Trong khi đó NoSQL lưu trữ dữ liệu trên rất nhiều loại cấu trúc dữ liệu khác nhau như đã bàn bên trên như dạng key-value, document hay graph.

Lược đồ dữ liệu của thực thể (Schema) Trong SQL thì mỗi bản ghi có schema là cố định, tức là mỗi column trong table phải được định nghĩa từ trước khi tạo bảng, và mỗi khi dữ liệu được thêm vào một row thì các column của nó phải có giá trị (chấp nhận giá trị NULL). Các schema có thể được thay đổi sau đó (alter table), nhưng sự thay đổi này phải nằm ở phía Database và khi DB thực hiện thay đổi này nó sẽ offline tạm thời. Trong NoSQL, schema là động (dynamic) có nghĩa là ta không cần phải định nghĩa một schema nào trước mà schema sẽ được dựa vào cấu trúc của records được đưa vào.

Truy vấn (Querying): SQL truy vấn thông qua một ngôn ngữ truy vấn mang tính cấu trúc được gọi là Structured Query Language (SQL) thao tác với dữ liệu. SQL là một ngôn ngữ rất mạnh mẽ và lâu đời là đại diện duy nhất cho toàn bộ các RDBMS khác nhau. NoSQL truy vấn của nó thường được gọi là UnSQL (Unstructured Query

Language) mỗi dạng NoSQL khác nhau sẽ có cấu trúc cú pháp khác nhau để có thể lấy ra các dạng dữ liệu khác nhau

Khả năng mở rộng (Scalability): Trong hầu hết các RDBMS thì khả năng mở rộng là theo chiều ngang (vertically scalable) ví dụ như tăng sức mạnh phần cứng của máy chủ hiện tại (tăng CPU, Ram...), tất nhiên rằng cách làm này rất đắt đỏ cũng như tốn thời gian mà đặc biệt là nó luôn có giới hạn nào đó. Còn với NoSQL DB thì hỗ trợ rất tốt cho mở rộng theo chiều dọc (horizontally scalable), có nghĩa là đơn giản chỉ thêm server vào các nodes hiện có, cách làm này đơn giản cũng như rẻ tiền hơn. Phần lớn các NoSQL DB công việc này đã được hỗ trợ sẵn và được làm tự động, có nghĩa là rất dễ để triển khai.

Độ tin cậy (Reliability): Hầu hết các RDBMS đều tuân thủ theo các thuộc tính của ACID (atomicity, consistency, isolation, và durability) vì vậy nó rất đảm bảo độ tin cậy và bảo toàn dữ liệu. Còn với NoSQL thì đa phần sẽ hy sinh tính chất ACID để đánh đổi với performance (hiệu năng) và khả năng mở rộng (scalability).

SQL hay NoSQL đều có những tính chất khác nhau phục vụ cho những nhu cầu đặc biệt khác nhau, do vậy không có DB nào thực sự là hoàn hảo phù hợp cho mọi nhu cầu tùy vào mục đích của hệ thống sở tại. Ngay cả khi hiện tại NoSQL đang trở nên phổ biến rộng rãi về tốc độ và khả năng mở rộng, nhưng vẫn có tình huống mà một SQL DB sẽ hoạt động tốt hơn (ví dụ hệ thống về giao dịch tiền tệ cần đảm bảo về tính ACID). Việc lựa chọn hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) thường được ưu tiên cho các hệ thống yêu cầu sự chặt chẽ về cấu trúc và độ tin cậy cao. Cụ thể, đối với các ứng dụng nghiệp vụ đòi hỏi xử lý giao dịch phức tạp, việc tuân thủ triệt để bốn thuộc tính ACID (Atomicity, Consistency, Isolation, Durability) là điều kiện tiên quyết để bảo toàn tính nhất quán và toàn vẹn dữ liệu, đặc biệt trong các lĩnh vực nhạy cảm như tài chính ngân hàng hoặc thương mại điện tử. Bên cạnh đó, SQL là giải pháp tối ưu khi lược đồ dữ liệu của hệ thống đã được xác định rõ ràng ngay từ giai đoạn thiết kế ban đầu. Mô hình này phục vụ tốt cho các nghiệp vụ có tính ổn định cao, ít yêu cầu thay đổi về cấu trúc trong tương lai và đặt trọng tâm vào tính nhất quán của dữ liệu hơn là khả năng mở rộng linh hoạt. Ngược lại, đối với các hệ thống hiện đại phải đối mặt với thách thức về dữ liệu lớn

và yêu cầu phát triển linh hoạt, NoSQL lại được xem là sự lựa chọn phù hợp hơn. Hệ quản trị này thể hiện sự ưu việt khi xử lý lượng lớn dữ liệu phi cấu trúc hoặc bán cấu trúc với tốc độ tăng trưởng nhanh nhờ cơ chế lưu trữ đa dạng, chẳng hạn như dạng document. Điểm mạnh cốt lõi của NoSQL nằm ở khả năng mở rộng theo chiều ngang (horizontally scalable), cho phép hệ thống dễ dàng nâng cao hiệu suất thông qua việc bổ sung các node vào cụm máy chủ thay vì phải nâng cấp phần cứng đơn lẻ, giúp giải quyết bài toán về lưu trữ dữ liệu quy mô lớn một cách hiệu quả. Xét về khía cạnh quy trình phát triển phần mềm, NoSQL đặc biệt tương thích với các mô hình phát triển nhanh (Rapid Development) như Agile. Nhờ đặc tính phi lược đồ (schema-less) hoặc lược đồ linh hoạt, công nghệ này cho phép đội ngũ kỹ thuật thay đổi cấu trúc dữ liệu giữa các chu kỳ phát triển mà không gây gián đoạn dịch vụ hay đòi hỏi các thao tác di trú dữ liệu phức tạp. Điều này giải quyết được hạn chế của RDBMS trong việc thích ứng với các yêu cầu nghiệp vụ chưa rõ ràng hoặc thường xuyên thay đổi, giúp giảm thiểu độ trễ và rủi ro khi triển khai các phiên bản mới. Cuối cùng, trong bối cảnh kiến trúc điện toán đám mây (Cloud Computing) ngày càng phổ biến, việc lưu trữ và phân tán dữ liệu trên nhiều máy chủ để đảm bảo tính sẵn sàng và khả năng mở rộng là yếu tố then chốt. Các hệ thống NoSQL được thiết kế tối ưu cho môi trường phân tán này, cho phép tự động hóa quá trình phân mảnh và nhân bản dữ liệu giữa các máy chủ. Nhờ đó, hệ thống có thể tận dụng tối đa lợi thế về chi phí và tài nguyên của hạ tầng đám mây mà không đòi hỏi sự can thiệp thủ công quá nhiều vào việc quản trị vận hành.

CHƯƠNG 2. VỀ CẤU TRÚC LƯU TRỮ DỮ LIỆU

2.1. Những điểm nổi bật trong cấu trúc lưu trữ dữ liệu

Cơ sở dữ liệu dạng Key-Value (Key-Value Store) đại diện cho một mô hình lưu trữ phi quan hệ (NoSQL) với kiến trúc tối giản, trong đó dữ liệu được tổ chức dưới dạng các cặp khóa - giá trị (key-value pairs). Về mặt bản chất, mô hình này vận hành tương tự như cấu trúc bảng băm phân tán (distributed hash table) hoặc cấu trúc từ điển, nơi mỗi khóa đóng vai trò là một định danh duy nhất (unique identifier) để truy xuất dữ liệu. Giá trị được lưu trữ gắn liền với khóa có thể tồn tại dưới nhiều định dạng đa dạng, từ các kiểu dữ liệu nguyên thủy như chuỗi ký tự, số, cho đến các cấu trúc phức tạp như đối tượng JSON, hình ảnh hoặc dữ liệu nhị phân.

Đặc điểm nổi bật của mô hình này là tính chất phi lược đồ (schema-less), cho phép hệ thống loại bỏ sự ràng buộc về cấu trúc bảng cố định thường thấy trong các hệ quản trị cơ sở dữ liệu quan hệ (RDBMS), từ đó tạo điều kiện thuận lợi cho việc lưu trữ linh hoạt các dữ liệu không đồng nhất. Về mặt hiệu năng, kiến trúc của Key-Value store được thiết kế để tối ưu hóa tối đa cho các tác vụ truy xuất dữ liệu thông qua khóa chủ. Nhờ việc ứng dụng các kỹ thuật băm (hashing) hoặc phân vùng dữ liệu, các thao tác đọc và ghi thường đạt được độ phức tạp thuật toán lý tưởng là $O(1)$, giúp giảm thiểu độ trễ và tăng tốc độ phản hồi. Chính vì vậy, loại hình cơ sở dữ liệu này đặc biệt phù hợp với các ứng dụng yêu cầu hiệu suất cao và xử lý thời gian thực, điển hình như hệ thống bộ nhớ đệm (caching), quản lý phiên làm việc (session management) hoặc các dịch vụ yêu cầu truy cập dữ liệu tức thời. Xét về khả năng mở rộng và độ tin cậy, các hệ thống Key-Value hiện đại thường hỗ trợ mạnh mẽ khả năng mở rộng theo chiều ngang (horizontal scaling). Điều này cho phép nâng cao năng lực lưu trữ và xử lý của hệ thống bằng cách bổ sung thêm các node vào cụm máy chủ (cluster) mà không làm gián đoạn hoạt động hay ảnh hưởng đến hiệu năng tổng thể. Đồng thời, thông qua các cơ chế phân mảnh dữ liệu (partitioning) và sao lưu tự động (replication), hệ thống đảm bảo được tính sẵn sàng cao (high availability) và khả năng chịu lỗi (fault tolerance), giúp bảo toàn dữ liệu ngay cả khi xảy ra sự cố phần cứng. Tuy nhiên, sự đơn giản trong kiến trúc cũng đi kèm với những hạn chế nhất định về khả năng truy vấn. Đặc trưng cơ bản của Key-Value store

là dữ liệu chỉ có thể được truy xuất chính xác thông qua khóa; hệ thống không hỗ trợ các truy vấn phức tạp như tìm kiếm theo nội dung giá trị, lọc theo điều kiện hay thực hiện các phép kết nối (join) giữa các tập dữ liệu như trong SQL. Hơn nữa, do giá trị thường được hệ thống xem như một khối dữ liệu mờ (blob) mà không cần hiểu cấu trúc bên trong, việc quản lý tính toàn vẹn, định dạng và kiểm soát chất lượng dữ liệu sẽ được chuyển giao hoàn toàn cho tầng ứng dụng xử lý. Do đó, Key-Value store là giải pháp tối ưu cho các trường hợp sử dụng có tải trọng lớn, mô hình dữ liệu đơn giản và ít mối quan hệ ràng buộc, nhưng không thay thế hoàn toàn được RDBMS trong các nghiệp vụ đòi hỏi truy vấn phức tạp.

Mô hình lưu trữ Key-Value thể hiện ưu thế vượt trội thông qua kiến trúc tối giản, giúp đơn giản hóa đáng kể quy trình thiết kế và vận hành hệ thống. Nhờ loại bỏ các ràng buộc về lược đồ dữ liệu (schema), mô hình này cho phép lưu trữ linh hoạt đa dạng các định dạng từ văn bản, số liệu đến các đối tượng phức tạp như JSON hay dữ liệu nhị phân, đáp ứng tốt nhu cầu của các ứng dụng có dữ liệu thay đổi liên tục. Đặc biệt, hiệu suất truy xuất là điểm mạnh cốt lõi khi các thao tác đọc/ghi dựa trên khóa được tối ưu hóa để đạt tốc độ phản hồi cực nhanh. Điều này khiến Key-Value store trở thành lựa chọn lý tưởng cho các tác vụ đòi hỏi xử lý thời gian thực, quản lý phiên làm việc hoặc đóng vai trò là bộ nhớ đệm (cache) trong các hệ thống như Redis hay Hazelcast, giúp giảm tải áp lực cho cơ sở dữ liệu backend nhờ tận dụng ưu điểm của bộ nhớ trong (in-memory). Bên cạnh hiệu năng, khả năng mở rộng theo chiều ngang (horizontal scaling) là một đặc tính quan trọng giúp hệ thống Key-Value thích ứng tốt với môi trường điện toán đám mây và hệ phân tán. Kiến trúc này cho phép các nhà quản trị dễ dàng bổ sung thêm các node vào cụm máy chủ để nâng cao năng lực xử lý mà không gây ảnh hưởng đến tài nguyên hiện hữu hay đòi hỏi thời gian gián đoạn dịch vụ (downtime). Sự linh hoạt trong việc mở rộng quy mô giúp hệ thống duy trì sự ổn định ngay cả khi phải đối mặt với tải trọng lớn hoặc sự gia tăng đột biến của dữ liệu. Tuy nhiên, việc áp dụng mô hình này đòi hỏi sự cân nhắc kỹ lưỡng về những hạn chế cố hữu trong khả năng truy vấn và quản lý mối quan hệ dữ liệu. Do cơ chế chỉ hỗ trợ truy xuất thông qua khóa định danh, hệ thống hoàn toàn thiếu vắng các khả năng truy vấn phức tạp như tìm kiếm theo

nội dung, lọc điều kiện hoặc thực hiện các phép kết nối (join) đặc trưng của RDBMS. Hệ quả là gánh nặng xử lý các logic phức tạp và đảm bảo tính toàn vẹn dữ liệu bị chuyển dịch về phía tầng ứng dụng, gây tốn kém chi phí và công sức phát triển. Đồng thời, mô hình này cũng không phù hợp cho các bài toán yêu cầu quản lý chặt chẽ các mối liên kết phức tạp giữa các đối tượng dữ liệu. Một thách thức kỹ thuật khác nằm ở chiến lược thiết kế khóa và quản lý tài nguyên phần cứng. Việc lựa chọn khóa không hợp lý trong môi trường phân tán có thể dẫn đến hiện tượng phân mảnh dữ liệu không đều (hotspot), gây mất cân bằng tải và suy giảm hiệu năng cục bộ. Hơn nữa, đối với các giải pháp lưu trữ in-memory như Redis hay Memcached, chi phí vận hành có thể tăng cao khi khối lượng dữ liệu lớn (Big Data) đòi hỏi dung lượng RAM khổng lồ, đặt ra bài toán kinh tế cần giải quyết giữa hiệu suất và chi phí hạ tầng.

2.2. Dữ liệu dạng Key values

2.2.1. World simplest database (Log structure)

Cơ sở dữ liệu đơn giản nhất thế giới (World simplest database) hoạt động dựa trên nguyên tắc lưu trữ dạng log (append-only log), tức là một kho lưu trữ key-value mà mọi thao tác ghi (db_set) chỉ đơn giản là ghi tuần tự (append) cặp khóa-giá trị vào cuối một tệp dữ liệu. Phương pháp này mang lại ưu điểm là hiệu suất ghi dữ liệu (db_set) rất tốt vì việc ghi thêm vào cuối tệp là một thao tác rất hiệu quả. Tuy nhiên, nhược điểm lớn nhất của nó là hiệu suất truy vấn kém (db_get) khi cơ sở dữ liệu có lượng bản ghi lớn. Để tìm kiếm giá trị mới nhất của một khóa, hệ thống phải quét toàn bộ tệp từ đầu đến cuối (chi phí $O(n)$) để tìm lần xuất hiện cuối cùng của khóa đó, làm chậm đáng kể quá trình đọc. Đây là lý do tại sao các hệ thống thực tế cần sử dụng các cấu trúc dữ liệu bổ sung như chỉ mục (index) để tăng tốc độ truy vấn.

```
#!/bin/bash

db_set () {
    echo "$1,$2" >> database
}

db_get () {
    grep "^$1," database | sed -e "s/^$1, //" | tail -n 1
}
```

Hình 2.1. Mô hình cơ sở dữ liệu cơ bản

Hình minh họa một mô hình cơ sở dữ liệu key–value tối giản được xây dựng bằng Bash script, trong đó dữ liệu được lưu trữ trực tiếp dưới dạng văn bản trong một tệp tin có tên database. Hệ thống cung cấp hai thao tác cơ bản gồm ghi dữ liệu (db_set) và truy xuất dữ liệu (db_get). Cụ thể, hàm db_set thực hiện việc lưu một cặp khóa–giá trị bằng cách ghi thêm một dòng mới theo định dạng “key,value” vào cuối tệp, thể hiện cách tiếp cận lưu trữ dạng append-only tương tự như cơ chế nhật ký (log). Trong khi đó, hàm db_get thực hiện truy vấn bằng cách tìm các dòng khớp với khóa yêu cầu, loại bỏ phần khóa khỏi kết quả và trả về giá trị được ghi gần nhất, nhờ đó đảm bảo hệ thống ưu tiên dữ liệu cập nhật mới nhất. Mặc dù cách triển khai này đơn giản và dễ hiểu, nó cho thấy rõ nguyên lý nền tảng của các hệ thống lưu trữ key–value trong thực tế, đồng thời gợi mở các vấn đề về hiệu năng khi kích thước dữ liệu tăng lên, do thao tác truy vấn vẫn phụ thuộc vào việc quét tuần tự tệp tin.

```
$ db_set 123456 '{"name":"London","attractions":["Big Ben","London Eye"]}'  
  
$ db_set 42 '{"name":"San Francisco","attractions":["Golden Gate Bridge"]}'  
  
$ db_get 42  
{"name":"San Francisco","attractions":["Golden Gate Bridge"]}
```

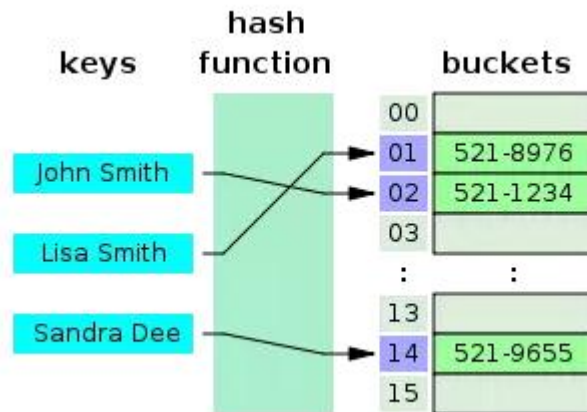
Hình 2.2. Lưu trữ và truy xuất trong mô hình cơ sở dữ liệu cơ bản

Ví dụ trong hình minh họa một kịch bản ứng dụng điển hình của mô hình lưu trữ dữ liệu dạng key–value, trong đó mỗi bản ghi được quản lý thông qua một khóa định danh duy nhất và giá trị tương ứng có thể biểu diễn dưới dạng dữ liệu có cấu trúc. Cụ thể, hệ thống sử dụng thao tác db_set để ghi dữ liệu bằng cách ánh xạ các khóa số như 123456 và 42 với các đối tượng JSON mô tả thông tin địa lý (ví dụ tên thành phố) và danh sách địa điểm nổi bật. Sau khi dữ liệu được lưu trữ, thao tác db_get được sử dụng để truy xuất giá trị dựa trên khóa đầu vào; trong trường hợp minh họa, truy vấn với khóa 42 trả về đối tượng JSON chứa dữ liệu về San Francisco và điểm tham quan Golden Gate Bridge. Thí nghiệm đơn giản này cho thấy mô hình key–value có khả năng lưu trữ linh hoạt đối với dữ liệu phi cấu trúc hoặc bán cấu trúc, đồng thời hỗ trợ cơ chế truy vấn trực tiếp theo khóa, giúp tối ưu hóa tốc độ truy xuất trong nhiều hệ thống lưu trữ hiện đại như NoSQL database hoặc in-memory cache. Tuy nhiên, hiệu năng tổng thể của hệ

thông phụ thuộc đáng kể vào cơ chế tổ chức và tìm kiếm khóa, đặc biệt khi quy mô dữ liệu tăng lên và số lượng bản ghi được ghi vào tệp lưu trữ ngày càng lớn.

2.2.2. Hash index

Chỉ mục băm (Hash Index) là một kỹ thuật tổ chức dữ liệu tiên tiến, được xây dựng dựa trên cấu trúc bảng băm (Hash Table) nhằm tối ưu hóa hiệu suất cho các truy vấn tìm kiếm chính xác. Khác với cấu trúc cây B-Tree vốn ưu thế về truy vấn phạm vi, Hash Index tập trung vào việc giảm thiểu chi phí truy xuất xuống mức thấp nhất, với độ phức tạp lý tưởng đạt mức $O(1)$. Cơ chế vận hành cốt lõi của hệ thống này dựa trên sự tương tác giữa mảng các bucket – đơn vị lưu trữ cơ bản chứa dữ liệu hoặc con trỏ – và hàm băm (Hash Function). Hàm băm đóng vai trò là công cụ ánh xạ toán học, chịu trách nhiệm chuyển đổi khóa tìm kiếm thành một địa chỉ cụ thể trong mảng bucket. Khi một truy vấn được thực hiện, hệ thống sẽ tính toán trực tiếp vị trí của dữ liệu thay vì phải duyệt tuần tự, từ đó mang lại tốc độ truy xuất vượt trội. Dựa trên khả năng thích ứng với sự thay đổi quy mô dữ liệu, kỹ thuật băm được phân loại thành băm tĩnh và băm động.

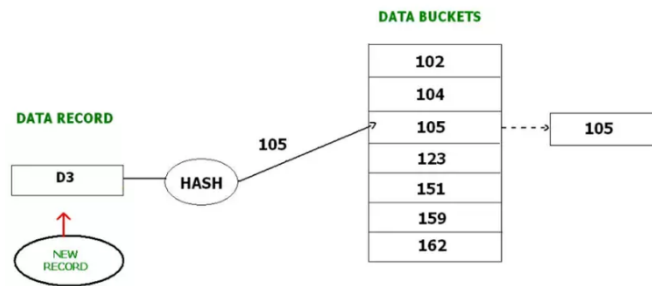


Hình 2.3. Cách Hash Table lưu và tra cứu dữ liệu

Hình ... minh họa mô hình hoạt động của bảng băm (hash table) trong việc tổ chức và truy xuất dữ liệu hiệu quả. Cụ thể, hệ thống bắt đầu từ tập các khóa (keys) ở phía bên trái, đại diện cho những định danh duy nhất như “John Smith”, “Lisa Smith” và “Sandra Dee”. Các khóa này không được lưu trực tiếp theo dạng tuyến tính, mà trước tiên được đưa qua hàm băm (hash function)—một cơ chế biến đổi khóa thành một giá trị số nguyên hoặc chỉ mục (index). Giá trị chỉ mục này đóng vai trò như “địa chỉ” để

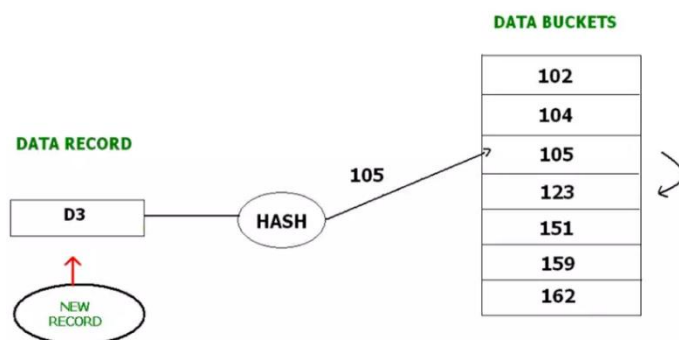
xác định vị trí lưu trữ tương ứng trong cấu trúc dữ liệu. Sau khi được băm, mỗi khóa được ánh xạ đến một bucket (ô chứa dữ liệu) nằm trong vùng bộ nhớ được đánh số, ví dụ như 00, 01, 02, 03,..., 13, 14, 15. Tại mỗi bucket, hệ thống lưu trữ giá trị (value) đi kèm với khóa, chẳng hạn như các số điện thoại “521-8976”, “521-1234” hoặc “521-9655”. Cách tổ chức này cho phép quá trình tra cứu diễn ra nhanh chóng: thay vì phải duyệt qua toàn bộ danh sách khóa, hệ thống chỉ cần áp dụng hàm băm lên khóa cần tìm để xác định trực tiếp bucket tương ứng, từ đó truy xuất giá trị gần như tức thời. Vì vậy, hash table được xem là một trong những cấu trúc dữ liệu quan trọng nhằm tối ưu hóa hiệu năng truy vấn, đặc biệt trong các ứng dụng cần truy cập dữ liệu thường xuyên với tốc độ cao.

Trong mô hình băm tĩnh (Static Hashing), số lượng bucket được cố định ngay từ đầu, dẫn đến thách thức về xử lý xung đột – hiện tượng hai khóa khác nhau cùng ánh xạ vào một vị trí. Để giải quyết vấn đề này, phương pháp Băm mở (Open Hashing) hay Liên kết chuỗi (Separate Chaining) thường được áp dụng. Kỹ thuật này sử dụng danh sách liên kết tại mỗi bucket để lưu trữ các bản ghi bị xung đột. Mặc dù phương pháp này cho phép số lượng phần tử vượt quá kích thước bảng băm mà không gây tràn, nhưng chi phí truy xuất có thể tăng tuyến tính nếu chuỗi liên kết trở nên quá dài. Ngược lại với phương pháp mở, kỹ thuật Băm đóng (Closed Hashing), hay còn gọi là Địa chỉ mở (Open Addressing), duy trì toàn bộ dữ liệu ngay trong không gian của bảng băm chính mà không sử dụng vùng nhớ phụ. Khi vị trí bucket được chỉ định đã bị chiếm giữ, hệ thống sẽ kích hoạt các thuật toán thăm dò như thăm dò tuyến tính hoặc bậc hai để tìm kiếm bucket trống tiếp theo. Ưu điểm của băm đóng là tiết kiệm bộ nhớ do không cần quản lý các con trỏ liên kết, tuy nhiên, hiệu suất của hệ thống sẽ suy giảm nghiêm trọng khi hệ số tải (load factor) tăng cao, đòi hỏi sự quản lý chặt chẽ về dung lượng. Tuy nhiên, hạn chế cố hữu của mô hình băm tĩnh là sự thiếu linh hoạt khi đối mặt với sự gia tăng nhanh chóng của dữ liệu, thường dẫn đến yêu cầu băm lại (rehash) toàn bộ hệ thống rất tốn kém.



Hình 2.4. Quá trình chèn (insert) một bản ghi mới vào cấu trúc băm mở

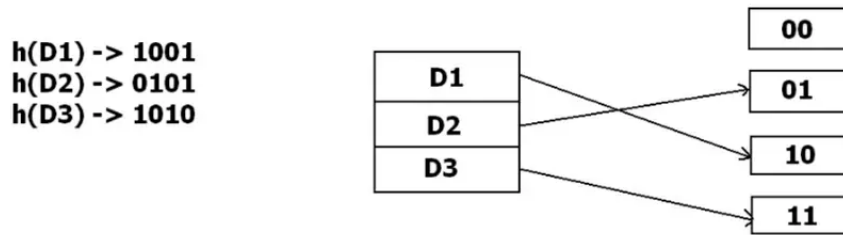
Hình minh họa cơ chế open hashing (còn gọi là separate chaining) trong quá trình lưu trữ một bản ghi dữ liệu mới bằng kỹ thuật băm. Cụ thể, khi một bản ghi mới (ký hiệu D3) được đưa vào hệ thống, trước tiên khóa của bản ghi sẽ được xử lý thông qua hàm băm (hash function) nhằm tạo ra một giá trị băm dùng để định vị vị trí lưu trữ. Trong trường hợp này, kết quả băm thu được là 105, tương ứng với một bucket (ô dữ liệu) trong tập các vùng lưu trữ được đánh số như 102, 104, 105, 123, 151, 159 và 162. Hệ thống sau đó truy cập trực tiếp đến bucket mang chỉ số 105 để thực hiện thao tác chèn dữ liệu. Trong mô hình open hashing, mỗi bucket không chỉ chứa một phần tử duy nhất mà có thể quản lý một danh sách các phần tử cùng ánh xạ đến bucket đó, nhờ đó cho phép xử lý hiệu quả hiện tượng xung đột băm (hash collision) khi nhiều khóa khác nhau tạo ra cùng một giá trị băm. Do đó, thay vì phải tìm một ô trống trong bảng như các phương pháp dò tìm tuyến tính, open hashing duy trì cấu trúc liên kết tại mỗi bucket để lưu nhiều bản ghi, góp phần đảm bảo tính linh hoạt trong lưu trữ và duy trì hiệu năng truy xuất ổn định trong các hệ thống dữ liệu có quy mô lớn.



Hình 2.5. Quá trình chèn (insert) một bản ghi mới vào cấu trúc băm đóng

Hình minh họa cơ chế băm đóng (closed hashing), hay còn gọi là open addressing, trong quá trình chèn một bản ghi dữ liệu mới vào bảng băm. Trong mô hình này, một bản ghi mới ký hiệu D3 được đưa vào hệ thống và trước tiên sẽ được xử lý thông qua hàm băm (hash function) nhằm tạo ra giá trị băm dùng để xác định vị trí lưu trữ ban đầu. Kết quả băm thu được trong trường hợp minh họa là 105, tương ứng với một bucket cụ thể trong cấu trúc lưu trữ gồm các địa chỉ như 102, 104, 105, 123, 151, 159 và 162. Khác với băm mở (open hashing) sử dụng danh sách liên kết để lưu nhiều phần tử trong cùng một bucket, băm đóng yêu cầu mỗi bucket chỉ lưu tối đa một phần tử. Do đó, khi xảy ra hiện tượng xung đột băm (hash collision)—tức vị trí bucket 105 đã có dữ liệu—hệ thống sẽ không tạo thêm vùng lưu trữ phụ mà thay vào đó sẽ tiến hành dò tìm (probing) để xác định một bucket trống khác theo một chiến lược định sẵn như dò tuyến tính (linear probing), dò bậc hai (quadratic probing) hoặc băm kép (double hashing). Cách tiếp cận này giúp bảng băm duy trì cấu trúc gọn trong cùng một vùng nhớ liên tục, đồng thời hỗ trợ truy xuất nhanh trong điều kiện tải hợp lý; tuy nhiên, hiệu năng có thể suy giảm khi hệ số tải tăng cao do hiện tượng cụm hóa (clustering), làm tăng số lần dò tìm cần thiết trong thao tác chèn và tìm kiếm.

Để khắc phục những nhược điểm của băm tĩnh, băm động (Dynamic Hashing) đã ra đời, cho phép không gian địa chỉ thay đổi linh hoạt theo nhu cầu thực tế. Các kỹ thuật như Băm mở rộng (Extendible Hashing) hay Băm tuyến tính (Linear Hashing) sử dụng cơ chế tách bucket thông minh hoặc cấu trúc thư mục để quản lý. Khi một bucket bị tràn, hệ thống chỉ phân chia lại bucket đó hoặc một nhóm nhỏ liên quan và cấp phát thêm không gian mới, thay vì phải cấu trúc lại toàn bộ cơ sở dữ liệu, qua đó đảm bảo hiệu suất ổn định cho các hệ thống quy mô lớn.



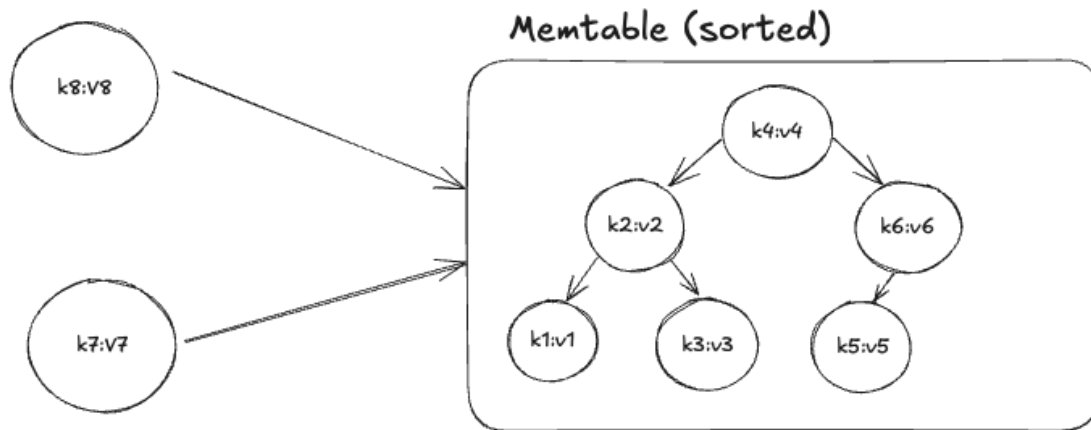
Hình 2.6. Quá trình chèn (insert) một bản ghi mới vào cấu trúc băm động

Hình minh họa cơ chế băm động (dynamic hashing), một phương pháp tổ chức và mở rộng cấu trúc băm nhằm duy trì hiệu năng truy xuất ổn định khi dữ liệu tăng dần theo thời gian. Trong mô hình này, mỗi bản ghi dữ liệu (ví dụ D1, D2, D3) sau khi được đưa vào hệ thống sẽ được xử lý bởi hàm băm để tạo ra một chuỗi bit đại diện cho giá trị băm, chẳng hạn như $h(D1)=1001$, $h(D2)=0101$ và $h(D3)=1010$. Thay vì ánh xạ trực tiếp toàn bộ chuỗi bit này vào một vị trí cố định như trong băm tĩnh, băm động sử dụng một phần tiền tố (prefix) của giá trị băm để định tuyến bản ghi vào các bucket tương ứng. Ở giai đoạn đầu, hệ thống có thể chỉ sử dụng 1 bit đầu để phân chia dữ liệu thành hai nhóm bucket 0 và 1, giúp cấu trúc đơn giản và tiết kiệm bộ nhớ khi số lượng dữ liệu còn nhỏ. Khi khối lượng bản ghi tăng lên và phát sinh xung đột hoặc tràn bucket, hệ thống sẽ mở rộng bằng cách tăng số bit được sử dụng (ví dụ lên 2 bit đầu) để tạo ra nhiều bucket hơn như 00, 01, 10, 11, nhờ đó phân phối lại dữ liệu một cách chi tiết hơn và giảm xác suất xung đột. Cơ chế mở rộng linh hoạt này cho phép băm động thích ứng với sự gia tăng dữ liệu mà không cần tái băm toàn bộ tập dữ liệu ngay từ đầu, qua đó cải thiện khả năng mở rộng (scalability) và đảm bảo hiệu năng trong các hệ thống lưu trữ hoặc cơ sở dữ liệu có yêu cầu cập nhật liên tục.

2.2.3. SSTable và LSMtree

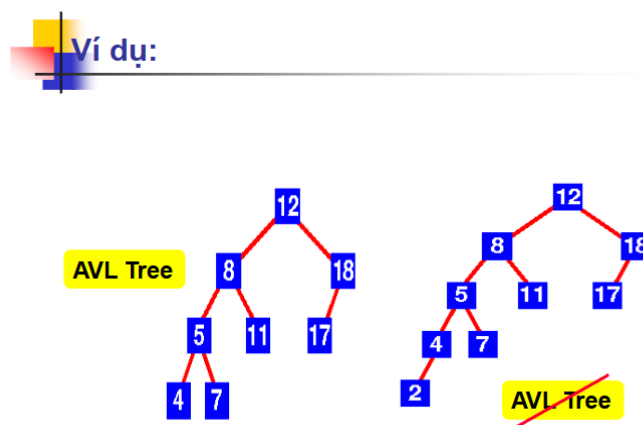
2.2.3.1. Cách LSM tree hoạt động

Khi có một yêu cầu ghi dữ liệu, thêm nó vào một cấu trúc dữ liệu **cây cân bằng** trong bộ nhớ (ví dụ như **cây đỏ-đen**). Cây trong bộ nhớ này được gọi là **memtable** (bảng dữ liệu được lưu trữ tạm thời trong RAM).



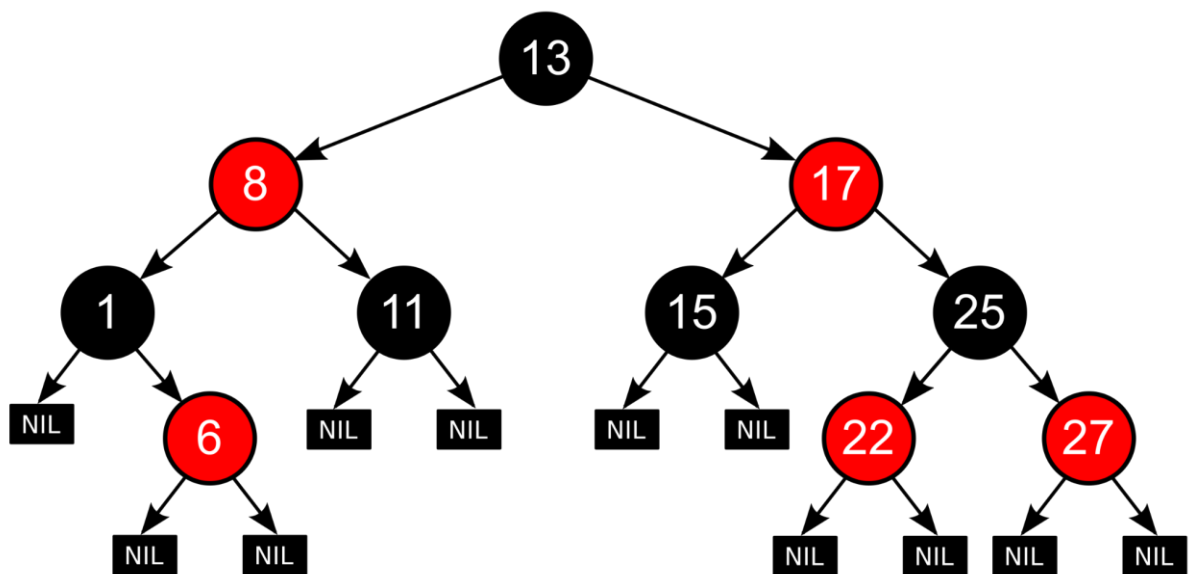
Hình 2.7. Minh họa cấu trúc memtable

- Cây cân bằng: cấu trúc dữ liệu dạng **cây nhị phân** (một nút có tối đa hai nút con) mà trong đó **chiều cao cây** (cân bằng tương đối)/**số nút con** (cân bằng hoàn toàn) tính từ node hiện tại của cây con bên trái và cây con bên phải hơn kém nhau không quá 1 đơn vị.



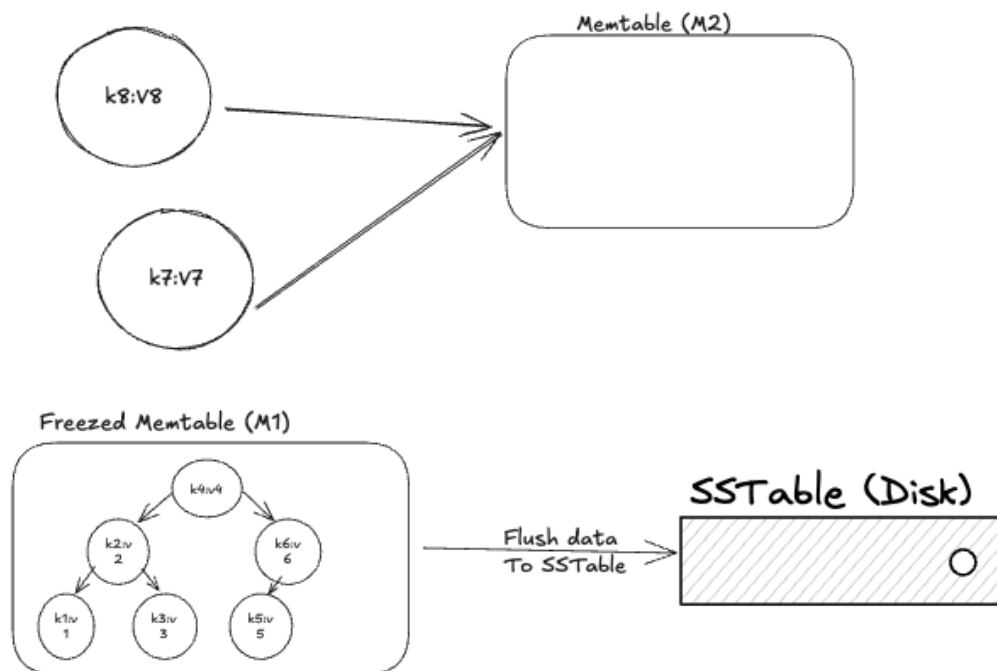
Hình 2.8. Minh họa cấu trúc cây nhị phân cân bằng AVL

- Cây đỏ đen (RB-tree): Một cấu trúc cây cân bằng thỏa các tính chất sau:
 - Một nút hoặc là đỏ hoặc là đen.
 - Gốc là đen.
 - Tất cả các lá (NULL) là đen.
 - Cả hai con của mọi nút đỏ là đen. (và suy ra mọi nút đỏ có nút cha là đen)
 - Tất cả các đường đi từ một nút bất kỳ tới các lá có số nút đen bằng nhau.



Hình 2.9. Cấu trúc của một cây đỏ đen (RB-tree)

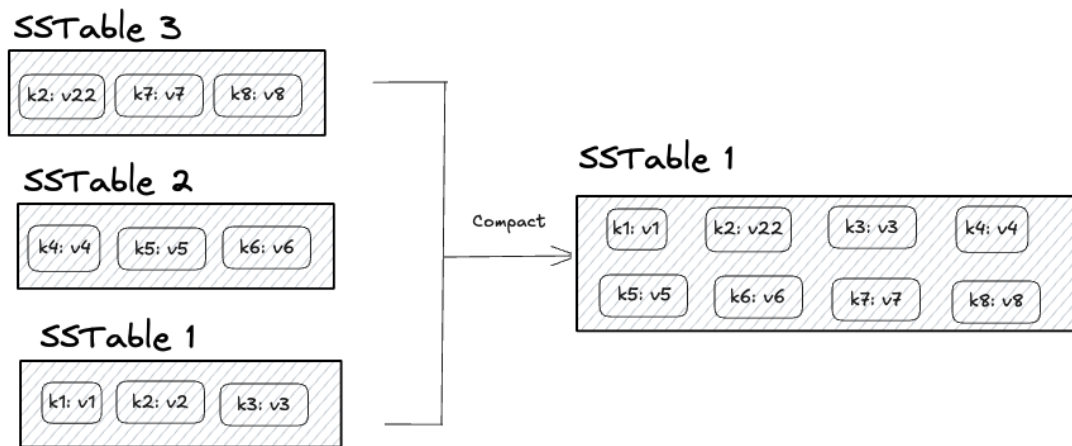
Khi memtable trở nên lớn hơn ngưỡng kích thước cho trước, ta sẽ lưu nó xuống Disk dưới định dạng **SSTable** và tạo **LSM-Tree Index** đi kèm và cây nhị phân cũng được hỗ trợ cho việc duyệt từ giá trị nhỏ đến lớn. SSTable mới được tạo sẽ trở thành segment gần đây nhất của DB, trong lúc đó thì chương trình sẽ tiếp tục ghi ra 1 memtable mới.



Hình 2.10. Minh họa quá trình memtable được lưu xuống đĩa

- SSTable: sorted string table, một file chứa dữ liệu dưới dạng key-value và được sắp xếp theo thứ tự key. Đây là file bất biến (đã ghi vào rồi thì không thay đổi nữa, chỉ đọc). Khi cần thay đổi dữ liệu của key thì giá trị mới của key sẽ nằm trong một sstable mới.

Để tìm kiếm 1 Key trong DB, đầu tiên ta tìm trong memtable hiện tại, nếu không thấy thì tìm tới segment gần đây nhất, cho tới các segment cũ hơn.



Hình 2.11. Minh họa quá trình nén các SSTable

Định kỳ theo thời gian, chạy 1 tiến trình **compaction** để gom các file segment và lọc bỏ đi các giá trị đã bị xóa/ghi đè.

2.2.3.2. Khi nào nên sử dụng LSM-tree

Vì những ưu điểm bên dưới nên LSM-tree cực kỳ phù hợp cho những hệ thống lớn, cần ghi cực nhiều dữ liệu ví dụ như **Cassandra** hay **ScyllaDB**.

2.2.3.3. Ưu và nhược điểm

Ưu điểm: Ghi dữ liệu nhanh (trên memtable), dễ mở rộng thêm dữ liệu → the SSTable sau đó compaction sẽ dọn dẹp sau).

Nhược:

- Mất nhiều thời gian trong việc truy vấn khi key không có trong db (do kiểm tra tuần tự từ mới đến cũ).
- Nếu DB bị crash, những giá trị gần nhất nằm trong memtable sẽ bị mất. Để giải quyết vấn đề đó, ta ghi vào append-only log trên disk. Mỗi memtable sẽ có file log riêng, nó được lưu theo thứ tự insert, thay vì được sắp xếp theo key vì nó chỉ dùng cho mục đích khôi phục DB. Ngay khi memtable được ghi xuống SSTable, ta có thể xóa file log cũ đi. [1]

2.2.4. B-Tree

2.2.4.1. Cách B-tree hoạt động

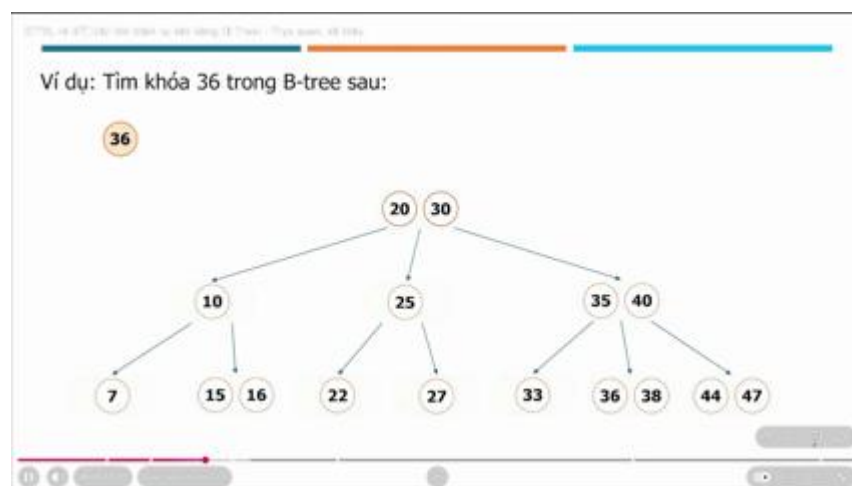
Khi có yêu cầu ghi dữ liệu, tương tự như LSM-tree B-tree vẫn giữ cấu trúc key-value tuy nhiên thay vì ghi tuần tự dữ liệu xuống file khi đạt kích thước cố định, B-tree chia cơ sở dữ liệu thành các **block** hoặc **pages** (đơn vị lưu trữ trên đĩa cứng) có kích thước cố định, thường là 4 Kb và mỗi block được định nghĩa theo cơ chế cây tham chiếu tương tự như con trỏ nhưng được áp dụng trên đĩa thay vì trên bộ nhớ.

Các đặc trưng của B-tree: cây được xác định bởi tham số m - bậc của cây.

- Số khóa của nút: tối đa $m-1$ khóa, tối thiểu $(m/2)-1$ khóa (trừ nút gốc).
- Số con của nút: tối đa m con, tối thiểu $(m/2)$ con (trừ nút gốc).
- Tính cân bằng: tất cả các nút lá nằm trên cùng một mức.
- Nút gốc: có ít nhất 1 khóa và 2 nút con (trừ khi cây rỗng)

Một pages được chỉ định làm gốc (root) của B-tree, khi tra cứu sẽ bắt đầu đi từ gốc và tiến hành tham chiếu lần lượt qua các nhánh đến các nút lá. [2]

a. Tìm khóa trong B-tree



Hình 2.12. Hình minh họa quá trình tìm khóa trong B-tree

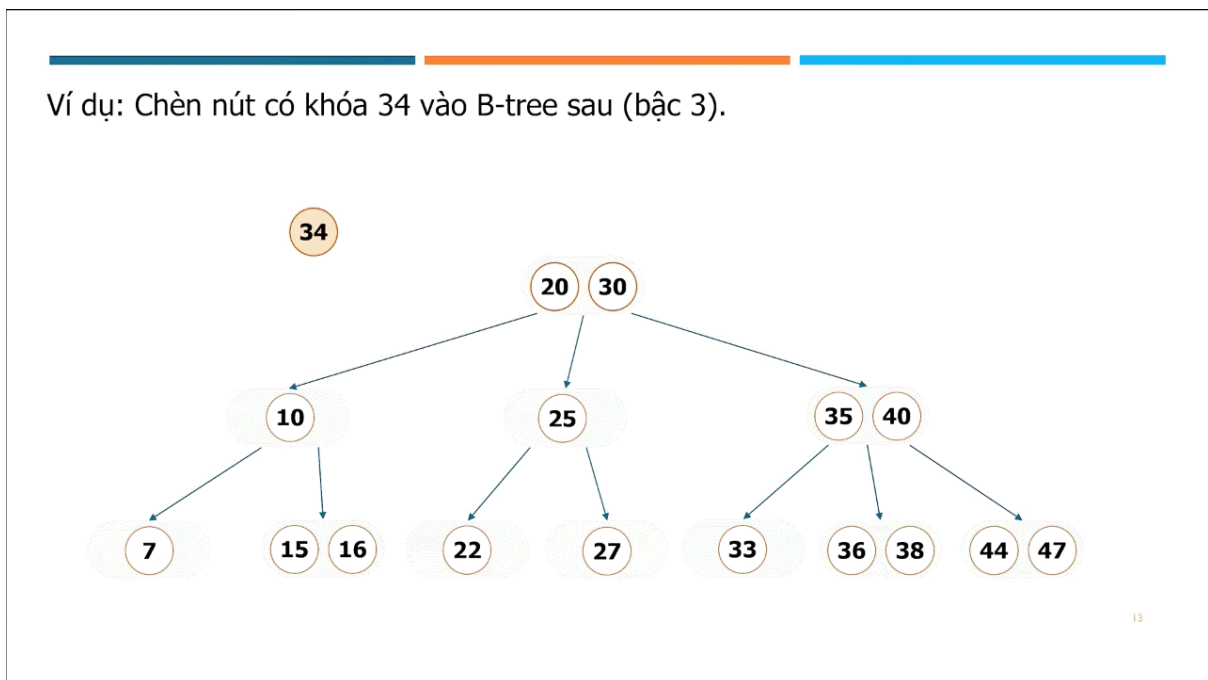
Trong hình là một cấu trúc B-tree bậc $m=3$ với m phụ thuộc vào quy định kích thước trang được cho trước.

Tìm khóa 36, đi từ node gốc. Ta thấy nút gốc có 2 khóa là 20 và 30, vì khóa 36 lớn hơn cả 20 và 30 nên ta tham chiếu đến nhánh con ngoài cùng bên phải vì trong b-tree thứ tự các khóa cũng được sắp xếp theo key giống như SSTable.

Tiếp tục tham chiếu lần lượt cho đến khi tìm thấy khóa 36 ở nút lá, lúc này ta có thể truy xuất giá trị của khóa 36 cần tìm.

Do đã được sắp xếp thứ tự các khóa nên ta dễ dàng xác định được khóa cần tìm có tồn tại hay không. Ví dụ, nếu muốn tìm khóa 50, ta cũng tham chiếu lần lượt từ gốc → nhánh con ngoài cùng bên phải → các nút lá ngoài cùng bên phải. các nút có khóa lớn nhất hiện tại là 47, hoàn toàn có thể xác định được là khóa cần tìm không tồn tại.

b. Chèn khóa trong B-tree



Hình 2.13. Hình minh họa quá trình chèn khóa mới vào B-tree

Dựa trên các nguyên tắc chung như trên, khi chèn khóa mới ta cần tham chiếu xem khóa này sẽ nằm ở nút lá nào. Nút lá có đầy hay chưa, nếu đã đầy thì đẩy phần tử khóa ở giữa lên nút cha và tách nút hiện tại ra làm đôi.

Cứ thực hiện lần lượt như vậy đến khi đây đạt đến giới hạn, không thể chèn thêm khóa nào nữa thì tiến hành tham chiếu → chèn khóa → lan truyền ngược lên trên → tăng bậc của cây.

c. Xóa khóa trong B-tree

Trường hợp 1: Xóa khóa tại nút lá:

Bước 1: Tìm và xóa khóa.

Bước 2: Kiểm tra số khóa còn lại trong nút.

Nếu nút vẫn có đủ số khóa tối thiểu ($\geq (m/2) - 1$), xong.

Nếu nút có ít khóa hơn mức tối thiểu, ta phải thực hiện tái cân bằng:

- Trường hợp có Thẻ Mượn (Redistribution/Borrowing)
- Trường hợp phải Hợp nhất (Merging)

Thao tác Mượn

Chuyển khóa tách biệt từ node cha xuống node lá bị thiếu.

Đẩy khóa nhỏ nhất của node anh/em bên phải hoặc khóa lớn nhất của node anh/em bên trái (nếu có thẻ mượn) lên làm khóa tách biệt mới trong node cha.

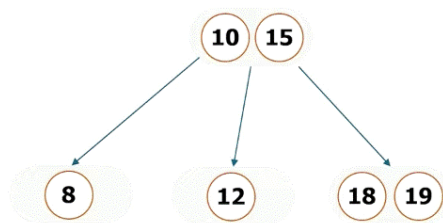
Thao tác Hợp nhất

Nếu các nút anh em cũng chỉ có số khóa tối thiểu, ta hợp nhất nút hiện tại, nút anh em và một khóa từ nút cha thành một nút duy nhất. Thao tác này có thể tiếp tục "lan truyền" lên trên nếu nút cha cũng bị thiếu khóa.

THUẬT TOÁN XÓA KHÓA KHỎI B - TREE

Trường hợp 1: Xóa khóa tại nút lá

Ví dụ: Xóa khóa 12 khỏi cây B dưới đây.



137

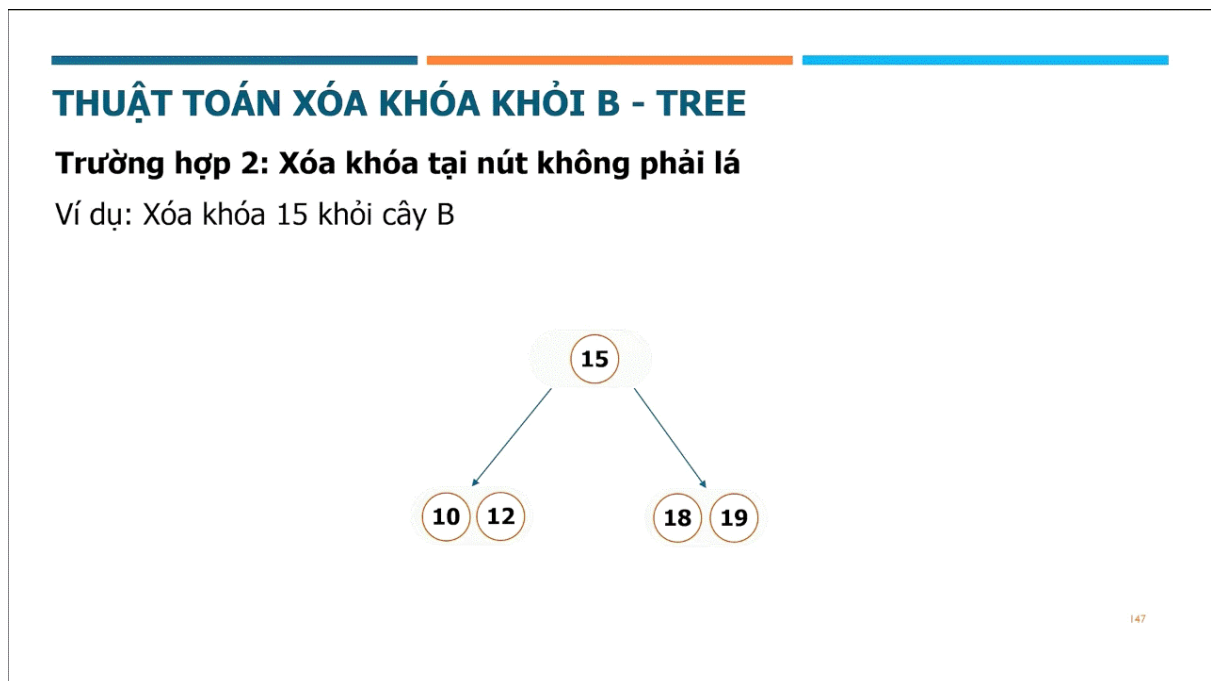
Hình 2.14. Minh họa quá trình xóa khóa tại nút lá

Trường hợp 2: Xóa khóa tại nút không phải lá

Bước 1: Tìm và xóa khóa.

Bước 2: Thay thế khóa đã xóa bằng khóa lớn nhất của cây con bên trái hoặc khóa nhỏ nhất của cây con bên phải.

Bước 3: Xóa khóa đã thay thế đó ở nút lá (vị trí ban đầu của nó). Thao tác này sẽ trở về trường hợp 1.



Hình 2.15. Minh họa quá trình xóa khóa tại nút không phải lá

Mỗi một nút (node) logic chúng ta hình dung trên cây này giờ sẽ được lưu thành một block hay một page trên đĩa cứng.

Khi cần sửa giá trị, ta tiến hành tham chiếu đến khóa và tiến hành cập nhật giá trị mới. Điều này có nghĩa là b-tree tiến hành chỉnh sửa trực tiếp trên khóa đó mà không phải ghi ra file mới.

2.2.4.2. Khi nào nên sử dụng B-tree

B-Tree thiết lập các chỉ mục cây rất chặt chẽ, giúp tối ưu hóa tốc độ truy vấn rất phù hợp cho các ứng dụng yêu cầu tốc độ truy vấn cao như MySQL (InnoDB) và PostgreSQL

2.2.4.3. Ưu và nhược điểm

Ưu:

- Tối ưu hóa I/O đĩa: Ưu điểm lớn nhất là giảm thiểu số lần truy cập đĩa (Disk I/O). Bằng cách lưu trữ nhiều khóa và con trỏ trong một nút (tương ứng với một khối đĩa vật lý), giúp tìm kiếm nhanh chóng ngay cả với tập dữ liệu khổng lồ.
- Hiệu suất tìm kiếm nhất quán: Do tính chất luôn tự cân bằng hoàn hảo (tất cả các nút lá đều ở cùng một độ sâu), thời gian cho các thao tác tìm kiếm, chèn, và xóa là ổn định và có độ phức tạp thời gian logarit.
- Hỗ trợ truy vấn phạm vi (Range Queries): Vì các khóa trong B-tree được lưu trữ có thứ tự, việc truy xuất dữ liệu trong một phạm vi giá trị cụ thể tương đối hiệu quả.
- Khả năng mở rộng (Scalability): B-tree có thể xử lý hiệu quả lượng dữ liệu lớn đến hàng terabyte mà không bị giảm hiệu suất đáng kể, làm cho nó trở thành lựa chọn lý tưởng cho các hệ thống cơ sở dữ liệu.
- Sửa tại chỗ (In-place updates)

Nhược:

- Tốn kém khi cập nhật thường xuyên: Mỗi khi chèn hoặc xóa dữ liệu, B-tree có thể cần thực hiện các thao tác tách, hợp nhất và cân bằng lại phức tạp (overhead), làm giảm hiệu suất khi có tần suất ghi/sửa quá cao.
- Tốn không gian lưu trữ: So với việc lưu trữ dữ liệu không có chỉ mục, B-tree cần thêm không gian để lưu trữ cấu trúc cây và các con trỏ.
- Độ phức tạp cài đặt: Logic để cài đặt và duy trì B-tree (đặc biệt là các thao tác xóa và cân bằng lại) phức tạp hơn đáng kể so với các cấu trúc dữ liệu cây đơn giản khác.

2.2.4.4. So sánh giữa LSM-tree và B-tree

Đặc điểm	B-Tree	LSM Tree (Log-Structured Merge Tree)
----------	--------	--------------------------------------

Tối ưu cho	Đọc (Read-heavy).	Ghi (Write-heavy).
Thao tác ghi	Ghi ngẫu nhiên (random write) và có thể sửa đổi dữ liệu tại chỗ trên đĩa, tốn nhiều I/O.	Ghi tuần tự (sequential write) vào Memtable (bộ nhớ), rất nhanh. Dữ liệu cũ bị đánh dấu xóa và hợp nhất sau (compaction).
Thao tác đọc	Nhanh, dữ liệu có sẵn ở một nút, ít truy cập đĩa.	Phức tạp hơn, có thể phải tìm ở nhiều cấp (memtable, SSTable khác nhau), tốn nhiều I/O hơn B-Tree.
Hiệu suất	Ghi chậm hơn, đọc nhanh hơn.	Ghi cực nhanh, đọc chậm hơn.
Sử dụng chính	Hệ quản trị cơ sở dữ liệu quan hệ (MySQL, PostgreSQL, SQL Server).	CSDL NoSQL (Cassandra, HBase, RocksDB, MongoDB WiredTiger).

Hình 2.16. Bảng so sánh LSM-tree và B-tree

2.3. Dữ liệu hướng cột (Column-oriented data)

2.3.1. Lưu trữ dữ liệu hướng cột là gì

Trong khi hầu hết các cơ sở dữ liệu quan hệ truyền thống lưu trữ dữ liệu theo hàng (row-oriented) — nghĩa là tất cả các giá trị của một hàng được lưu trữ cạnh nhau trên đĩa — thì lưu trữ hướng cột lưu trữ tất cả các giá trị của một cột cụ thể cạnh nhau. Thay vì một tệp chứa các dòng (Name, Age, Department), sẽ có các tệp riêng biệt: một tệp chỉ chứa tất cả Name, một tệp chỉ chứa tất cả Age, ...

Employee				
EmployeeId	FullName	CountryName	PhoneNumber	DepartmentGroupName
1	Lane nay	United States	257-555-0154	Quality Assurance
2	Pete C	United States	883-555-0116	Finance
3	Nuan Yu	Canada	922-555-0165	Finance
4	Candy Spoon	Canada	185-555-0169	Marketing
5	Jae Pak	United Kingdom	1 (11) 500 555-0145	Manufacturing
6	Ranjit Varkey	France	1 (11) 500 555-0117	Accounting

Hình 2.17. Minh họa cấu trúc dữ liệu truyền thống

Với bảng trên thì dữ liệu được ghi có dạng sau:

1: Lane nay 2: Pete C 3: Nuan Yu 4: Candy Spoon 5: Jae Pak 6: Ranjit Varkey
1, 2: United States 3, 4: Canada 5: United Kingdom 6: France
1: 257-555-0154 2: 883-555-0116 3: 922-555-0165 4: 185-555-0169 5: 1 (11) 500 555-0145 6: 1 (11) 500 555-0117
1: Quality Assurance 2, 3: Finance 4: Marketing 5: Manufacturing 6: Accounting

Hình 2.18. Minh họa cấu trúc dữ liệu hướng cột

Một ví dụ khác theo trong sách:

fact_sales table							
date_key	product_sk	store_sk	promotion_sk	customer_sk	quantity	net_price	discount_price
140102	69	4	NULL	NULL	1	13.99	13.99
140102	69	5	19	NULL	3	14.99	9.99
140102	69	5	NULL	191	1	14.99	14.99
140102	74	3	23	202	5	0.99	0.89
140103	31	2	NULL	NULL	1	2.49	2.49
140103	31	3	NULL	NULL	3	14.99	9.99
140103	31	3	21	123	1	49.99	39.99
140103	31	8	NULL	233	1	0.99	0.99

Columnar storage layout:

date_key file contents: 140102, 140102, 140102, 140102, 140103, 140103, 140103, 140103

product_sk file contents: 69, 69, 69, 74, 31, 31, 31, 31

store_sk file contents: 4, 5, 5, 3, 2, 3, 3, 8

promotion_sk file contents: NULL, 19, NULL, 23, NULL, NULL, 21, NULL

customer_sk file contents: NULL, NULL, 191, 202, NULL, NULL, 123, 233

quantity file contents: 1, 3, 1, 5, 1, 3, 1, 1

net_price file contents: 13.99, 14.99, 14.99, 0.99, 2.49, 14.99, 49.99, 0.99

discount_price file contents: 13.99, 9.99, 14.99, 0.89, 2.49, 9.99, 39.99, 0.99

Hình 2.19. Một ví dụ khác của cấu trúc dữ liệu hướng cột

2.3.2. Ưu điểm so với cơ chế lưu truyền thống

Giảm tải I/O: Trong các truy vấn phân tích (OLAP), thường chỉ cần đọc vài cột trong số hàng trăm cột của bảng. Lưu trữ hướng cột cho phép hệ thống chỉ đọc đúng những cột cần thiết từ đĩa, thay vì phải đọc toàn bộ hàng rồi mới lọc bỏ các cột dư thừa.

Tận dụng bộ nhớ đệm (Cache): Dữ liệu cùng loại được xếp liền kề nhau giúp tối ưu hóa việc sử dụng CPU cache và thực thi vector hóa (vectorized processing). [3]

Tiêu chí	Row-based storage	Column-based storage
Tổ chức dữ liệu	Mỗi hàng lưu trữ một bản ghi hoàn chỉnh, bao gồm tất cả các trường (hoặc cột) của bản ghi đó.	Dữ liệu được tổ chức thành các cột riêng lẻ. Mỗi cột chứa tất cả các giá trị của một trường cụ thể cho tất cả các hàng.
Truy cập dữ liệu	Đơn giản, vì bạn truy cập tất cả các cột của hàng đó cùng một lúc.	Phức tạp hơn, vì bạn cần ghép nối các cột để có bản ghi hoàn chỉnh.
Hoạt động giao dịch	Hiệu quả cho các hoạt động giao dịch, nơi bạn thường cần truy cập và sửa đổi toàn bộ bản ghi.	Không hiệu quả cho các hoạt động giao dịch.
Truy vấn phân tích	Không hiệu quả cho các truy vấn phân tích, vì đọc toàn bộ hàng có thể không cần thiết.	Hiệu quả cho các truy vấn phân tích, vì bạn chỉ cần truy cập các cột cụ thể.

Chi phí lưu trữ	Nếu không tối ưu hóa đúng cách thì chi phí lưu trữ dữ liệu cao.	Tiết kiệm không gian lưu trữ dữ liệu, vì các giá trị giống nhau trên một cột sẽ được gom nhóm lại để lưu trữ. Do đó, chi phí lưu trữ thấp.
------------------------	---	--

Hình 2.20. Bảng so sánh cấu trúc dữ liệu hướng hàng và hướng cột

2.3.3. Khi nào nên sử dụng dữ liệu hướng cột

Nhờ những điểm nổi bật trên nên lưu trữ hướng cột phù hợp cho các hệ thống OLAP, nơi cần tối ưu hóa cho việc tính toán, tổng hợp và truy vấn trên lượng dữ liệu khổng lồ dùng để phân tích. Cụ thể hơn, lưu trữ hướng cột rất thích hợp cho các ứng dụng phân tích dữ liệu như hệ thống báo cáo, khai thác dữ liệu, ...

2.3.4. Một số kỹ thuật trong lưu trữ hướng cột

2.3.4.1. Nén cột (Column compression)

Ngoài việc chỉ tải những cột cần thiết cho việc truy vấn, lưu trữ hướng cột còn giúp dễ dàng hơn trong việc giảm dung lượng bằng cách nén cột. Có thể thấy khi tổ chức dữ liệu dưới dạng cột thì các bản ghi trong cột đều có cùng kiểu dữ liệu và thường có những giá trị lặp lại. Có nhiều kỹ thuật nén cột khác nhau tùy thuộc vào dữ liệu trong cột nhưng một trong những kiểu phổ biến hiệu quả là kỹ thuật bitmap như hình minh họa bên dưới.

Column values:

product_sk:

69	69	69	69	74	31	31	31	31	29	30	30	31	31	31	68	69	69
----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

Bitmap for each possible value:

product_sk = 29:

0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 30:

0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 31:

0	0	0	0	0	1	1	1	1	0	0	0	1	1	1	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 68:

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 69:

1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	1	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

product_sk = 74:

0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Run-length encoding:

product_sk = 29:	9, 1	(9 zeros, 1 one, rest zeros)
product_sk = 30:	10, 2	(10 zeros, 2 ones, rest zeros)
product_sk = 31:	5, 4, 3, 3	(5 zeros, 4 ones, 3 zeros, 3 ones, rest zeros)
product_sk = 68:	15, 1	(15 zeros, 1 one, rest zeros)
product_sk = 69:	0, 4, 12, 2	(0 zeros, 4 ones, 12 zeros, 2 ones)
product_sk = 74:	4, 1	(4 zeros, 1 one, rest zeros)

Hình 2.21. Lưu trữ dữ liệu dạng nén cột được lập chỉ mục bằng bitmap.

Thông thường, số lượng giá trị khác nhau trong một cột nhỏ hơn nhiều so với số lượng hàng (ví dụ, một nhà bán lẻ có thể có hàng tỷ giao dịch bán hàng, nhưng chỉ có 100.000 sản phẩm khác nhau). Giờ đây, chúng ta có thể lấy một cột với n giá trị khác nhau và chuyển nó thành n bitmap riêng biệt: một bitmap cho mỗi giá trị khác nhau, với một bit cho mỗi hàng. Bit có giá trị 1 nếu hàng đó có giá trị đó, và 0 nếu không có.

Nếu n rất nhỏ (ví dụ, cột quốc gia có thể có khoảng 200 giá trị khác nhau), các bitmap đó có thể được lưu trữ với một bit trên mỗi hàng. Nhưng nếu n lớn hơn, sẽ có rất nhiều số 0 trong hầu hết các bitmap (chúng ta gọi đó là bitmap thừa). Trong trường hợp đó, các bitmap có thể được mã hóa theo độ dài chuỗi (run-length encoded), như được hiển thị ở cuối hình 2.20. Điều này có thể làm cho việc mã hóa một cột trở nên nhỏ gọn đáng kể.

2.3.4.2. Thứ tự sắp xếp (sort order)

Trong kiểu lưu trữ theo cột, thứ tự lưu trữ các hàng không nhất thiết quan trọng. Cách dễ nhất là lưu trữ chúng theo thứ tự được chèn vào, vì khi đó việc chèn một hàng mới chỉ đơn giản là thêm vào cuối mỗi tệp cột. Tuy nhiên, chúng ta có thể chọn áp đặt một thứ tự, giống như chúng ta đã làm với SSTable trước đây, và sử dụng điều đó như một cơ chế lập chỉ mục.

Việc sắp xếp từng cột một cách độc lập sẽ không có ý nghĩa, vì khi đó chúng ta sẽ không còn biết được mục nào trong các cột thuộc cùng một hàng. Chúng ta chỉ có thể tái tạo lại một hàng khi biết rằng mục thứ A trong một cột thuộc cùng một hàng với mục thứ A trong cột khác.

Thay vào đó, dữ liệu cần được sắp xếp toàn bộ một hàng tại một thời điểm, ngay cả khi nó được lưu trữ theo cột. Người quản trị cơ sở dữ liệu có thể chọn các cột mà bảng nên được sắp xếp, dựa trên kiến thức của họ về các truy vấn phổ biến. Ví dụ, nếu các truy vấn thường nhắm mục tiêu vào các khoảng thời gian, chẳng hạn như tháng trước, thì việc đặt `date_key` làm khóa sắp xếp đầu tiên có thể là hợp lý. Khi đó, trình tối ưu hóa truy vấn chỉ cần quét các hàng từ tháng trước, điều này sẽ nhanh hơn nhiều so với việc quét tất cả các hàng.

Cột thứ hai có thể xác định thứ tự sắp xếp của bất kỳ hàng nào có cùng giá trị ở cột đầu tiên. Ví dụ, nếu `date_key` là khóa sắp xếp đầu tiên trong Hình 3-10, thì việc sử dụng `product_sk` làm khóa sắp xếp thứ hai có thể hợp lý để tất cả các giao dịch bán hàng cho cùng một sản phẩm trong cùng một ngày được nhóm lại với nhau trong kho lưu trữ. Điều này sẽ giúp ích cho các truy vấn cần nhóm hoặc lọc giao dịch bán hàng theo sản phẩm trong một khoảng thời gian nhất định.

Một ưu điểm khác của việc sắp xếp theo thứ tự là nó có thể giúp nén các cột. Nếu cột sắp xếp chính không có nhiều giá trị khác nhau, thì sau khi sắp xếp, nó sẽ có các chuỗi dài trong đó cùng một giá trị được lặp lại nhiều lần liên tiếp. Một phương pháp mã hóa độ dài chuỗi đơn giản, như chúng ta đã sử dụng cho các bitmap trong Hình 3-11, có thể nén cột đó xuống chỉ còn vài kilobyte — ngay cả khi bảng có hàng tỷ hàng.

Hiệu ứng nén mạnh nhất ở khóa sắp xếp đầu tiên. Khóa sắp xếp thứ hai và thứ ba sẽ bị xáo trộn nhiều hơn, và do đó sẽ không có các chuỗi giá trị lặp lại dài như vậy. Các cột nằm ở vị trí thấp hơn trong thứ tự sắp xếp xuất hiện gần như ngẫu nhiên, vì vậy chúng có thể sẽ không được nén tốt bằng. Nhưng việc sắp xếp được một vài cột đầu tiên vẫn là một lợi thế tổng thể.

2.3.4.3. Cơ chế ghi (writing)

Những tối ưu hóa này rất hữu ích trong các kho dữ liệu, bởi vì phần lớn tải trọng bao gồm các truy vấn chỉ đọc lớn do các nhà phân tích thực hiện. Lưu trữ theo cột, nén và sắp xếp đều giúp tăng tốc các **truy vấn đọc** đó. Tuy nhiên, chúng lại có nhược điểm là làm cho việc **ghi dữ liệu trở nên khó khăn** hơn.

Phương pháp cập nhật tại chỗ, như cây B sử dụng, không khả thi với các cột được nén. Nếu bạn muốn chèn một hàng vào giữa một bảng đã được sắp xếp, rất có thể bạn sẽ phải viết lại tất cả các tệp cột. Vì các hàng được xác định bằng vị trí của chúng trong một cột, nên việc chèn phải cập nhật tất cả các cột một cách nhất quán.

Tất cả các thao tác ghi trước tiên đều được lưu trữ trong bộ nhớ, nơi chúng được thêm vào một cấu trúc được sắp xếp và chuẩn bị để ghi vào đĩa. Không quan trọng bộ nhớ lưu trữ được định hướng theo hàng hay theo cột. Khi đủ số lần ghi tích lũy, chúng được hợp nhất với các tệp cột trên đĩa và được ghi hàng loạt vào các tệp mới. Về cơ bản, đây là những gì **Vertica** thực hiện.

Các truy vấn cần kiểm tra cả dữ liệu cột trên đĩa và các thao tác ghi gần đây trong bộ nhớ, rồi kết hợp cả hai. Tuy nhiên, trình tối ưu hóa truy vấn lại che giấu sự khác biệt này khỏi người dùng. Từ góc nhìn của nhà phân tích, dữ liệu đã được sửa đổi bằng các thao tác chèn, cập nhật hoặc xóa sẽ được phản ánh ngay lập tức trong các truy vấn tiếp theo.

2.3.4.4. Khối dữ liệu và khung nhìn vật chất hóa (Data Cubes and Materialized Views)

Không phải mọi kho dữ liệu đều nhất thiết phải là kho lưu trữ theo cột: các cơ sở dữ liệu hướng theo hàng truyền thống và một vài kiến trúc khác cũng được sử

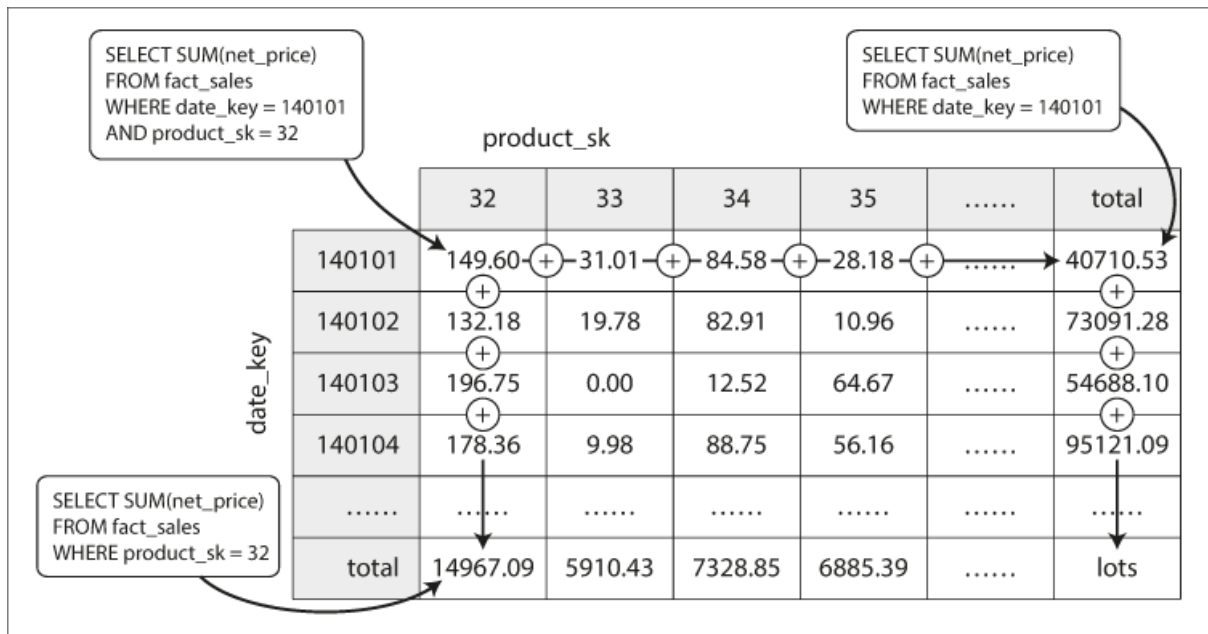
dụng. Tuy nhiên, lưu trữ theo cột có thể nhanh hơn đáng kể đối với các truy vấn phân tích, vì vậy nó đang nhanh chóng trở nên phổ biến.

Một khía cạnh khác của kho dữ liệu đáng được đề cập ngắn gọn là các tổng hợp được vật chất hóa. Như đã thảo luận trước đó, các truy vấn kho dữ liệu thường liên quan đến một hàm tổng hợp, chẳng hạn như COUNT, SUM, AVG, MIN hoặc MAX trong SQL. **Nếu cùng một hàm tổng hợp được sử dụng bởi nhiều truy vấn khác nhau, việc xử lý dữ liệu thô mỗi lần có thể gây lãng phí.** Tại sao không lưu trữ một số giá trị đếm hoặc tổng mà các truy vấn sử dụng thường xuyên nhất?

Một cách để tạo bộ nhớ đệm như vậy là sử dụng **materialized view (chế độ xem vật chất)**. Trong mô hình dữ liệu quan hệ, nó thường được định nghĩa giống như một chế độ xem chuẩn (ảo): **một đối tượng giống bảng có nội dung là kết quả của một số truy vấn.** Sự khác biệt là **materialized view là một bản sao thực sự của kết quả truy vấn, được ghi vào đĩa**, trong khi virtual view chỉ là một lối tắt để viết truy vấn. **Khi bạn đọc từ một virtual view, công cụ SQL sẽ mở rộng nó thành truy vấn cơ bản của view ngay lập tức và sau đó xử lý truy vấn đã được mở rộng.**

Khi dữ liệu gốc thay đổi, materialized view cần được cập nhật vì nó là bản sao phi chuẩn hóa của dữ liệu. Cơ sở dữ liệu có thể tự động thực hiện việc này, nhưng các bản cập nhật như vậy làm cho việc ghi dữ liệu tốn kém hơn, đó là lý do tại sao materialized view không thường được sử dụng trong các cơ sở dữ liệu OLTP. Trong các kho dữ liệu có nhiều thao tác đọc, chúng có thể mang lại nhiều ý nghĩa hơn (việc chúng có thực sự cải thiện hiệu suất đọc hay không phụ thuộc vào từng trường hợp cụ thể).

Một trường hợp **đặc biệt phổ biến của chế độ xem vật chất được gọi là khối dữ liệu hoặc khối OLAP.** Đó là một lưới các tập hợp được nhóm theo các chiều khác nhau. Hình bên dưới cho thấy một ví dụ.



Hình 2.22. Hai chiều của khối dữ liệu, tổng hợp dữ liệu bằng phép cộng.

Hãy tưởng tượng rằng mỗi sự kiện chỉ có khóa ngoại trỏ đến hai bảng chiều—trong hình 2.21, đó là ngày và sản phẩm. Bây giờ bạn có thể vẽ một bảng hai chiều, với ngày dọc theo một trục và sản phẩm dọc theo trục còn lại. Mỗi ô chứa tổng hợp (ví dụ: SUM) của một thuộc tính (ví dụ: net_price) của tất cả các bảng sự kiện có sự kết hợp ngày-sản phẩm đó. Sau đó, bạn có thể áp dụng cùng một phép tổng hợp cho mỗi hàng hoặc cột và nhận được một bản tóm tắt đã được giảm bớt một chiều (**doanh số theo sản phẩm bất kể ngày, hoặc doanh số theo ngày bất kể sản phẩm**).

Nhìn chung, các sự kiện thường có nhiều hơn hai chiều. Trong Hình 3-9 có năm chiều: ngày, sản phẩm, cửa hàng, chương trình khuyến mãi và khách hàng. Thật khó để hình dung một khối siêu lập phương năm chiều sẽ trông như thế nào, nhưng nguyên tắc vẫn giữ nguyên: **mỗi ô chứa doanh số bán hàng cho một sự kết hợp cụ thể giữa ngày-sản phẩm-cửa hàng-chương trình khuyến mãi-khách hàng**. Sau đó, các giá trị này có thể được tổng hợp lặp đi lặp lại dọc theo mỗi chiều.

Ưu điểm của khối dữ liệu vật chất hóa là một số truy vấn trở nên rất nhanh vì chúng đã được tính toán trước. Ví dụ, nếu bạn muốn biết tổng doanh số bán hàng của mỗi cửa hàng ngày hôm qua, bạn chỉ cần xem tổng số theo chiều phù hợp — không cần phải quét hàng triệu hàng.

Nhược điểm là khối dữ liệu không có tính linh hoạt như việc truy vấn dữ liệu thô. Ví dụ, không có cách nào để tính toán tỷ lệ doanh số đến từ các mặt hàng có giá hơn 100 đô la, vì giá cả không phải là một trong các chiều dữ liệu. Do đó, hầu hết các kho dữ liệu đều cố gắng giữ lại càng nhiều dữ liệu thô càng tốt và **chỉ sử dụng các tập hợp dữ liệu như khối dữ liệu như một biện pháp tăng hiệu suất cho một số truy vấn nhất định.** [4]

CHƯƠNG 3. OLTP và OLAP

3.1. OLTP (Online Transaction Processing)

3.1.1. Tổng quan về OLTP

OLTP hay còn gọi là Online Transaction Processing là các hệ thống được thiết kế chuyên biệt để xử lý các giao dịch dữ liệu hàng ngày của một tổ chức một cách nhanh chóng, chính xác và có trật tự.

Ban đầu, nó phục vụ các hoạt động thương mại như bán hàng, đặt hàng hoặc quản lý nhân viên,... nhưng sau đó khái niệm "giao dịch" (transaction) được mở rộng thành một đơn vị công việc logic bao gồm các thao tác như thêm, sửa, xóa hoặc một nhóm các thao tác có tính nguyên tử (ACID). Các hệ thống này hiện diện khắp nơi trong cuộc sống: từ máy ATM, hệ thống gửi tin nhắn điện thoại, cho đến máy tính tiền tại siêu thị.

OLTP được hình thành để giải quyết các mục đích cốt lõi như quản lý các giao dịch hàng ngày của các hoạt động kinh doanh thường nhật (như rút tiền, mua hàng, gửi tin nhắn) một cách mượt mà. Đảm bảo tính trật tự và an toàn dữ liệu của hệ thống, tránh sự tranh chấp dữ liệu, các người dùng khác nhau không can thiệp hoặc gây lỗi cho nhau. Hệ thống xây dựng niềm tin bằng cách đảm bảo các giao dịch dữ liệu của bạn không bị lẫn lộn với các giao dịch khác và không bị mất mát, nó còn thực hiện các giao dịch phi tài chính như đổi mật khẩu, gửi tin nhắn văn bản, quản lý nhân sự,...

3.1.2. Đặc điểm của OLTP

Hệ thống OLTP thường được xây dựng dựa trên nền tảng cơ sở dữ liệu quan hệ (Relational Database), dữ liệu thường được tổ chức lưu trữ theo dạng hàng và được chuẩn hóa để phục vụ cho việc thêm, sửa, xóa nhanh chóng. Nó tập trung vào việc xử lý các đơn vị công việc logic bao gồm các thao tác insert, update, delete hoặc một số nhóm các thao tác phải tuân thủ tính nguyên tử (ACID).

Một số đặc điểm chính của hệ thống OLTP:

- Write-heavy: Tập trung nhiều vào các thao tác ghi và cập nhật dữ liệu.
- Xử lý nhanh và chi tiết: Xử lý lượng bản ghi nhỏ trong mỗi giao dịch, các truy vấn thường đơn giản và sử dụng khóa chính (primary key) để truy xuất.

- Tính ACID (Atomic, Consistent, Isolated, Durable): Hệ thống phải đảm bảo đủ 4 yếu tố: Ổn định (không dễ thay đổi), độc lập, nhất quán và nguyên tử (không dễ phân chia).
- Hỗ trợ đa người dùng: Phục vụ người dùng cuối qua các ứng dụng web/mobile, cho phép nhiều người dùng truy cập cùng một thời điểm mà không gây xung đột dữ liệu trong hệ thống.

3.1.3. Ưu điểm và nhược điểm

3.1.3.1. Ưu điểm:

Tốc độ xử lý của hệ thống cao, được thiết kế để phản hồi cực nhanh cho các giao dịch đơn lẻ.

Hệ thống chính xác tuyệt đối, đảm bảo được tính toàn vẹn và nhất quán của dữ liệu (Data Consistency), dữ liệu luôn đúng tại mọi thời điểm mà người dùng truy xuất.

Hệ thống cho phép hàng ngàn người dùng cùng thao tác trên hệ thống tại cùng một thời điểm mà không gặp lỗi xung đột dữ liệu.

Hệ thống lưu trữ dữ liệu ở mức độ chi tiết nhất, giúp ghi lại chính xác lịch sử từng giao dịch.

3.1.3.2. Nhược điểm:

Khi người dùng cần sử dụng lượng lớn dữ liệu để lập báo cáo cũng như phân tích, thì việc thống kê phải đòi hỏi quét trên một lượng lớn dữ liệu rất rộng nhưng hệ thống lại không thể đáp ứng ngay lập tức cho yêu cầu này từ người dùng vì thế có thể gây chậm cho hệ thống.

Khi thực hiện các phép tính như SUM, AVERAGE, COUNT trên tập dữ liệu lớn có thể gặp khó khăn cho OLTP vì nó được thiết kế để xử lý chi tiết trên từng dòng, ví dụ khi người dùng chỉ thực hiện phân tích trên một số column thì việc này có thể gặp khó khăn đối với các hệ thống chứa hàng trăm hoặc thậm chí hàng triệu column vì OLTP cần phải tải toàn bộ column để xử lý trước khi trả về kết quả các column mà người dùng yêu cầu.

3.2. OLAP (Online Analytical Processing)

3.2.1. Tổng quan về OLAP

OLAP hay còn gọi là Online Analytical Processing. Đây là hệ thống được thiết kế chuyên biệt để nhằm phục vụ nhu cầu phân tích lượng lớn dữ liệu của một hệ thống, hỗ trợ phân tích chiều sâu và đa chiều của cơ sở dữ liệu. Khác với hệ thống OLTP được thiết kế để xử lý nhanh lượng lớn các giao dịch đơn lẻ, đảm bảo tính toàn vẹn cho dữ liệu, OLAP được thiết kế để quản lý cơ sở dữ liệu, lưu trữ và xử lý dữ liệu với khối lượng lớn, các giao dịch không phải là ưu tiên của chúng, thường được hướng đến lượng nhỏ người dùng, thường là các nhà phân tích, quản lý để phân tích để đưa ra giải pháp, hướng phát triển cho doanh nghiệp.

Ban đầu, các doanh nghiệp thường lưu trữ dữ liệu của họ vào các hệ thống OLTP, sau một thời gian vận hành doanh nghiệp, số lượng dữ liệu mà được lưu trữ ngày càng lớn, khi các nhà phân tích muốn phân tích, thống kê từ hệ thống cơ sở dữ liệu, họ thường tiến hành phân tích trên toàn bộ cơ sở dữ liệu hoặc phân tích trên một số thuộc tính (các column) nhưng hệ thống OLTP thường không đáp ứng được các yêu cầu này, dữ liệu được yêu cầu thường được trả về rất chậm vì OLTP được lưu trữ theo dạng hàng nên mỗi lần cần phân tích dữ liệu cần phải quét qua toàn bộ hệ thống. Vì thế, hệ thống OLAP được thiết kế nhằm giải quyết vấn đề này, các mô hình dữ liệu của OLAP như Star Schema hoặc Oriented-Column đã được thiết kế để khi người dùng yêu cầu phân tích trên một số thuộc tính, thì các mô hình này có thể xử lý nhanh và trả về kết quả khi chỉ xử lý trên những cột hoặc bảng dữ liệu liên quan và trả về kết quả nhanh chóng thay vì phải xử lý trên toàn bộ dữ liệu như OLTP.

OLAP ra đời để giải quyết những hạn chế mà hệ thống giao dịch (OLTP) không làm được, cụ thể là phục vụ các mục đích như phân tích dữ liệu lớn, đảm bảo nhu cầu xử lý và phân tích trên lượng lớn dữ liệu khổng lồ mà các hệ thống thông thường khó mà làm được. Hỗ trợ các nhà phân tích đưa ra quyết định chiến lược bằng cách cung cấp các thông tin để làm báo cáo, thống kê. Giúp người dùng có thể nhìn thấy được bức tranh toàn cảnh các điểm mấu chốt trong dữ liệu của họ. Hỗ trợ phân tích đa chiều, từ nhiều góc độ khác nhau.

3.2.2. Đặc điểm của OLAP

Hệ thống OLAP cũng có một số đặc điểm đặc trưng riêng của hệ thống như:

- Read-heavy: Tập trung chủ yếu vào thao tác đọc dữ liệu.
- Truy vấn phức tạp: Sử dụng các truy vấn tổng hợp như Sum, AVG, Count,... Các truy vấn này thường lớn và mất nhiều thời gian để thực hiện.
- Không yêu cầu thời gian thực: Không yêu cầu cập nhật tức thì hay xử lý thời gian thực như OLTP.
- Đối tượng sử dụng hẹp: Nhóm người dùng nhỏ hơn so với OLTP, chủ yếu là nhà quản lý, nhà phân tích hoặc các hệ thống của một doanh nghiệp
- Ưu tiên phân tích: Việc đảm bảo tính chính xác và toàn vẹn của từng giao dịch nhỏ lẻ (như OLTP) không phải là ưu tiên của OLAP.

3.2.3. Ưu điểm và nhược điểm

3.2.3.1. Ưu điểm:

Khả năng phân tích mạnh mẽ: Tối ưu hóa cho việc thống kê và phân tích từ đó có thể tìm ra xu hướng của dữ liệu phức tạp, giúp nhìn ra bức tranh toàn cảnh, tổng thể của doanh nghiệp.

Xử lý dữ liệu khổng lồ: Có khả năng làm việc với khối lượng dữ liệu rất lớn, các dữ liệu được tích trữ qua thời gian của các doanh nghiệp, đảm bảo hiệu quả của việc phân tích.

Hỗ trợ quyết định tốt: Cung cấp các số liệu tổng hợp (sum, count, average) chính xác để phục vụ báo cáo và chiến lược kinh doanh.

3.2.3.2. Nhược điểm:

Tốc độ phản hồi chậm với truy vấn: Các truy vấn trong OLAP thường lớn và mất nhiều thời gian để thực hiện, không nhanh chóng như các lệnh trong OLTP vì OLTP chỉ xử lý các giao dịch đơn lẻ và nhỏ gọn, còn OLAP đa số đều xử lý trên lượng lớn dữ liệu.

Không phù hợp cho giao dịch tức thời: Không được thiết kế để xử lý các tác vụ thêm/sửa/xóa (Insert/Update/Delete) nhanh chóng hay đảm bảo tính toàn vẹn của từng giao dịch nhỏ lẻ.

Đòi hỏi hạ tầng lớn: Hệ thống phải rất lớn và mạnh mẽ mới có thể lưu trữ và xử lý hết dữ liệu kinh doanh.

CHƯƠNG 4. DATA WAREHOUSE

4.1. Tổng quan về Data Warehouse

Data Warehouse là một hệ thống được thiết kế để lưu trữ, truy vấn và phân tích một lượng lớn dữ liệu từ nhiều nguồn khác nhau. Data Warehouse tập trung vào việc lưu trữ dữ liệu theo từng chủ đề và được tối ưu hóa cho việc truy vấn và phân tích dữ liệu, chứ không chỉ tập trung riêng cho giao dịch. Khác với các cơ sở dữ liệu thông thường chỉ phục vụ cho các giao dịch hàng ngày, Data Warehouse chuyên dùng để hỗ trợ phân tích dữ liệu và ra quyết định chiến lược. Dữ liệu trong Data Warehouse thường được tổ chức theo các bảng, cho phép người dùng dễ dàng thực hiện các truy vấn phức tạp để khai thác thông tin.

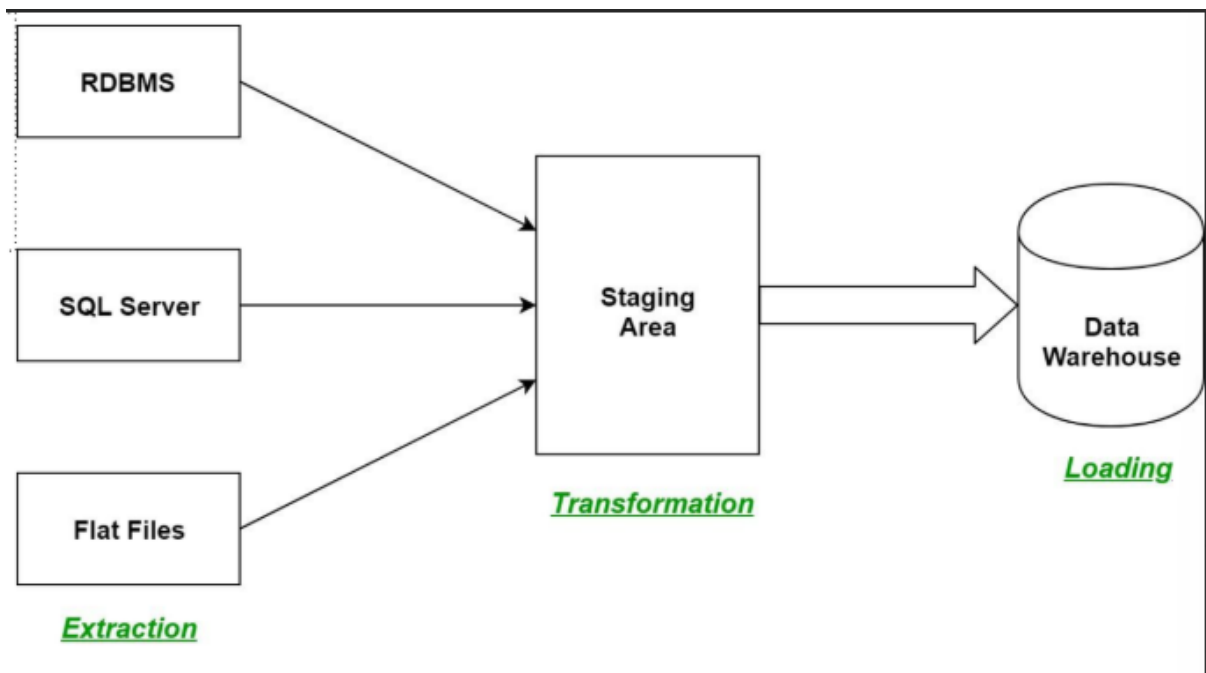
Data Warehouse đóng vai trò là trung tâm lưu trữ dữ liệu của doanh nghiệp, nên nó mang một số đặc điểm nổi bật so với các hệ thống cơ sở dữ liệu truyền thống như sau:

- **Hướng chủ đề (Subject-Oriented):** Data warehouse được thiết kế và tổ chức dựa trên các chủ đề hoặc lĩnh vực cụ thể của doanh nghiệp, dữ liệu sẽ được tổ chức và sắp xếp gọn gàng theo từng chủ đề cụ thể, chẳng hạn như bán hàng, marketing, tài chính, nhân sự, v.v. Kho dữ liệu sẽ tập trung vào việc cung cấp thông tin liên quan đến các chủ đề này để hỗ trợ quá trình ra quyết định.
- **Được tích hợp (Integrated):** Data warehouse tổng hợp và tích hợp dữ liệu từ nhiều nguồn khác nhau, bao gồm các hệ thống giao dịch và các nguồn dữ liệu bên ngoài, quá trình tích hợp này sẽ được thực hiện trong quá trình ETL (Extract - Transform - Load). Dữ liệu từ các nguồn này được chuẩn hóa và hợp nhất thành một cấu trúc dữ liệu chung, giúp tạo ra một bức tranh toàn diện về hoạt động của tổ chức.
- **Bất biến (Non - Volatile):** Dữ liệu trong data warehouse không thay đổi và chỉ hiện thị ở chế độ chỉ đọc (read-only). Thay vì cập nhật dữ liệu trực tiếp, các dữ liệu mới sẽ được ghi thêm vào và các dữ liệu đã lưu trước đó sẽ không bị xóa. Data warehouse thường được cung cấp dữ liệu từ các

nguồn gốc và duy trì lịch sử thay đổi của dữ liệu theo thời gian. Từ đó cho phép người dùng phân tích và so sánh dữ liệu theo các chu kỳ thời gian khác nhau. Data Warehouse tác biệt hoàn toàn với cơ sở dữ liệu hoạt động, khi có bất kỳ thay đổi nào thì sẽ không còn xuất hiện trong kho dữ liệu.

- Có gán nhãn thời gian (Time Variant): Data warehouse lưu trữ dữ liệu theo thời gian tại thời điểm nhập liệu và sẽ cho phép người dùng xem và phân tích dữ liệu theo các chu kỳ thời gian khác nhau, từ quá khứ đến hiện tại, giúp doanh nghiệp có thể so sánh các dữ liệu với nhau để đánh giá chúng đang phát triển theo chiều hướng tích cực hay tiêu cực.

ETL là viết tắt của cụm từ Extract, Transform, Load. Đây là một phần trung tâm của việc xây dựng và duy trì kho dữ liệu, dữ liệu từ nhiều nguồn khác nhau được rút trích, chuyển đổi để phù hợp với mô hình dữ liệu của kho dữ liệu và sau đó được tải lên kho dữ liệu. Kho dữ liệu sau đó cung cấp cấu trúc để phân tích dữ liệu và tạo báo cáo. Đầu tiên, dữ liệu được thu thập từ nhiều nguồn khác nhau, như từ cơ sở dữ liệu SQL, tệp CSV, trang web,... Sau đó, dữ liệu được chuẩn hóa và biến đổi để phù hợp với mô hình dữ liệu hoặc cơ sở dữ liệu, để đảm bảo tính nhất quán của dữ liệu. Và cuối cùng, sau khi dữ liệu đã được chuẩn bị và chuyển đổi, nó sẽ được tải vào kho dữ liệu hoặc cơ sở dữ liệu mục tiêu.

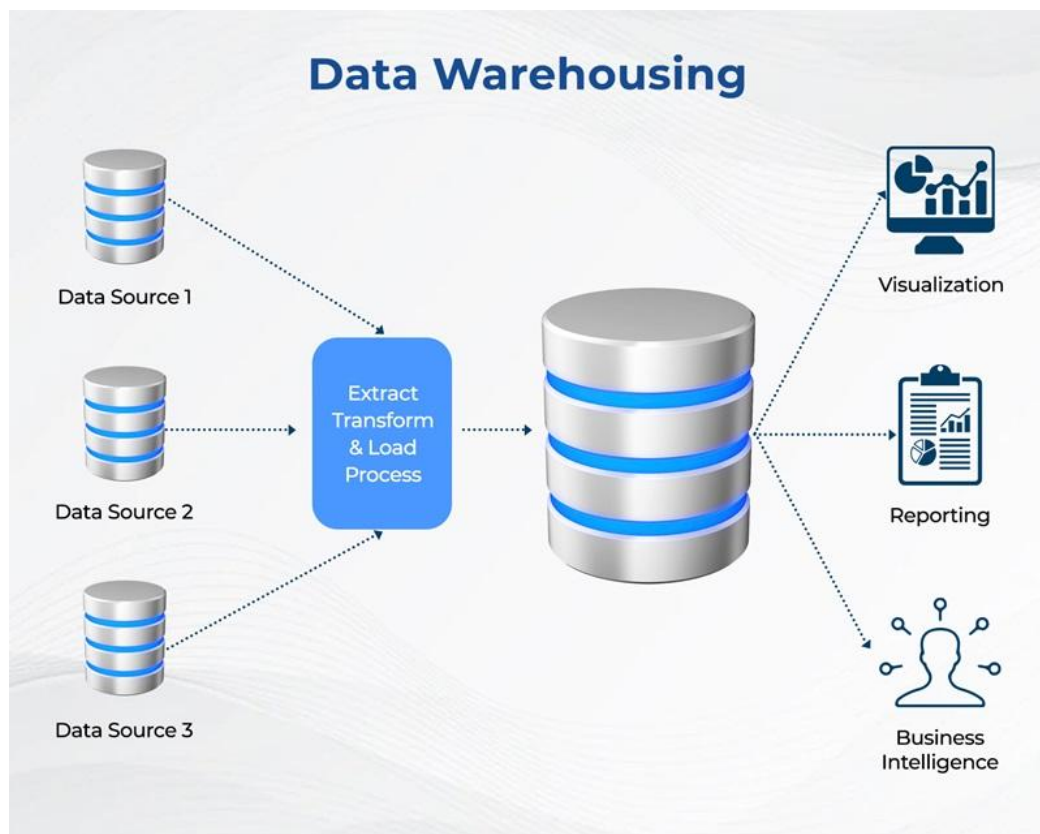


Hình 4.1. Minh họa về quy trình ETL

Tùy vào mục đích sử dụng cũng như quy mô thì Data Warehouse được phân thành 3 loại chính như sau:

- **Data Mart:** Là một tập hợp con của Data Warehouse được thu nhỏ hơn, được thiết kế để tập trung vào một phân khúc cụ thể của doanh nghiệp hoặc một nhóm người dùng cụ thể, đóng vai trò duy trì một khu vực/bộ phận hoặc đơn vị kinh doanh cụ thể. Từng bộ phận nhỏ của một doanh nghiệp có thể có một kho lưu trữ dữ liệu trung tâm riêng biệt. Data mart chứa dữ liệu đã được xử lý và tùy chỉnh để đáp ứng nhu cầu phân tích, được lưu định kỳ trong ODS và sau đó được gửi đến EDW.
- **Kho dữ liệu hoạt động (Operational Data Store – ODS):** Được coi như là một kho dữ liệu lưu trữ trung gian, nơi lưu trữ dữ liệu gần như theo thời gian thực từ các hệ thống giao dịch. ODS nhận dữ liệu từ nhiều nguồn khác nhau và cung cấp nhanh chóng các thông tin cập nhật, có thể giúp xử lý các dữ liệu ngắn hạn, hỗ trợ ra quyết định tức thời mà không cần xử lý toàn bộ dữ liệu trong quá khứ. Dữ liệu trong ODS cũng thường được làm nguồn cung cấp dữ liệu cho hệ thống EDW.

- Doanh nghiệp Kho Dữ liệu Doanh nghiệp (Enterprise Data Warehouse – EDW): là kho dữ liệu lớn nhất, bao quát toàn bộ tổ chức, được xây dựng để phục vụ cho việc lưu trữ dữ liệu cho toàn bộ doanh nghiệp. EDW thu thập dữ liệu từ nhiều nguồn trong đó có từ ODS và tổ chức chúng thành một cấu trúc dữ liệu chung để hỗ trợ người dùng khai thác và thực hiện các hoạt động phân tích ở cấp độ sâu hơn và phân tích từ nhiều chiều, từ đó có thể thực hiện các báo cáo và đưa ra các quyết định của tổ chức. [1],



Hình 4.2. Kiến trúc tổng quan của hệ thống Data Warehouse và quy trình dòng chảy dữ liệu

4.2. Kiến trúc của Data Warehouse

Simple: Là một kiến trúc cơ bản và đơn giản của Data Warehouse. Kiến trúc này trực tiếp thu thập dữ liệu từ nhiều nguồn sau đó sao chép vào Data Warehouse mà không có bất kỳ xử lý hay biến đổi dữ liệu nào, tất cả dữ liệu đều là dữ liệu thô, tất cả được lưu trong trung tâm dữ liệu. Điều này làm đơn giản hóa quy trình và tăng tốc độ truy xuất dữ liệu.

Simple with a staging area: Kiến trúc này có bao gồm thêm một khu vực tạm trữ (staging area) nằm giữa nguồn dữ liệu và Data Warehouse chính. Trong quá trình trích xuất và lưu trữ thông tin vào Data Warehouse, dữ liệu sẽ được sao chép và lưu trữ tạm thời vào khu vực staging area, cho phép kho dữ liệu kiểm tra, làm sạch và biến đổi dữ liệu trước khi dữ liệu được lưu trữ vào Data Warehouse chính.

Hub and Spoke: Kiến trúc này bao gồm một kho trung tâm lớn (Hub) được kết nối với các nguồn dữ liệu (Spoke). Các Hub sẽ là nơi được tổng hợp, xử lý và lưu trữ dữ liệu chung, trong khi các nguồn dữ liệu được giữ riêng tại các Spoke. Nhờ vậy mà kiến trúc này có tính linh hoạt cao và hiệu quả trong các môi trường phức tạp, vì thế đây là một kiến trúc phổ biến trong các hệ thống Data Warehouse lớn.

Sandboxes: Là một mô hình dữ liệu cho phép các nhóm hoặc người dùng tạo ra các môi trường độc lập để thực hiện phân tích và thử nghiệm dữ liệu. Mỗi sandbox là một không gian làm việc riêng biệt và có thể điều chỉnh phù hợp nhằm đáp ứng nhu cầu và yêu cầu cụ thể của từng nhóm. Điều này giúp đảm bảo tính riêng tư và an toàn của dữ liệu trong quá trình phân tích và thử nghiệm. [2], [3]

4.3. Chức năng của Data Warehouse

Lưu trữ dữ liệu: Data Warehouse cung cấp một hệ thống lưu trữ tập trung, nơi mà tất cả dữ liệu từ nhiều nguồn khác nhau được tổng hợp và sắp xếp một cách có hệ thống. Nhờ đó mà người dùng có thể quản lý dữ liệu của mình một cách hiệu quả.

Phân tích và tổng hợp dữ liệu: Data Warehouse có khả năng phân tích các dữ liệu đã tổng hợp, từ đó có thể thống kê và phán đoán xu hướng của dữ liệu.

Hỗ trợ ra quyết định: Nhờ khả năng tổng hợp và phân tích dữ liệu, Data Warehouse có thể giúp đưa ra các quyết định dựa vào xu hướng của dữ liệu.

Tổng hợp dữ liệu từ nhiều nguồn: Data Warehouse cho phép tích hợp dữ liệu từ nhiều nguồn khác nhau, các dữ liệu này được xử lý và tích hợp với kho dữ liệu, giúp đồng nhất dữ liệu tổng hợp.

4.4. Các mô hình dữ liệu kinh điển

4.4.1. Star Schema

4.4.1.1. Tổng quan về Star Schema

Star Schema là một dạng kiến trúc mô hình dữ liệu quan hệ có cấu trúc đơn giản nhất, trong đó dữ liệu được tổ chức thành các dữ liệu và các chiều. Cấu trúc của nó gồm hai thành phần chính, có một bảng dữ liệu fact (bảng thống kê) nằm ở trung tâm thiết kế, xung quanh bảng fact này là các bảng dimensions chứa các dữ liệu mang tính đối tượng (object) liên quan tới bảng fact.

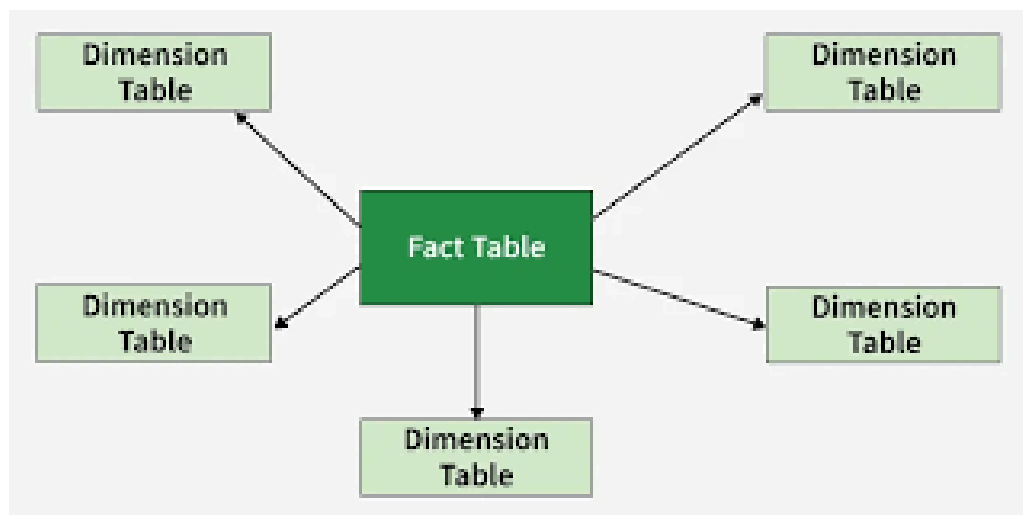
Star Schema là cách thiết kế đơn giản nhất trong Data Warehouse. Tuy nhiên, nó lại được sử dụng rất phổ biến vì cách thiết kế đơn giản và đặc tính tối ưu hóa của nó trong việc truy xuất dữ liệu.

Fact table hay bảng dữ kiện là thành phần trung tâm của một mô hình giản đồ hình sao, có chức năng lưu trữ các quan sát hoặc sự kiện kinh doanh cụ thể như đơn hàng, số dư kho, tỷ giá hối đoái hay nhiệt độ. Bảng Fact có hai loại cột chính đó là các cột dữ liệu định lượng (numeric measure columns) và các cột khóa ngoại (dimension key columns). Các cột khóa ngoại này liên kết trực tiếp với các bảng dimension, và tập hợp của các khóa ngoại thường được dùng làm khóa chính cho bảng Fact. Bảng Fact thường lưu trữ các sự kiện diễn ra liên tục, cho nên bảng Fact thường có kích thước rất lớn và không ngừng gia tăng số lượng hàng theo thời gian.

- Một đặc điểm quan trọng cần xem xét khi thiết kế bảng Fact là mức độ chi tiết (granularity) của dữ liệu. Các cột khóa chính xác định kích thước của bảng Fact, nhưng chính các giá trị của khóa mới quyết định mức độ chi tiết. Ví dụ, một bảng theo dõi mục tiêu bán hàng có thể bao gồm thời gian và mã sản phẩm, nếu giá trị trong cột thời gian được lưu là ngày đầu tiên của mỗi tháng, thì mức độ chi tiết của bảng sẽ ở cấp độ tháng chứ không phải ngày hay tuần. Dữ liệu trong bảng có thể được lưu trữ ở cấp độ chi tiết nhất hoặc ở dạng tổng hợp, nhưng thông thường các dữ kiện trong cùng một bảng sẽ có cùng một mức độ tổng hợp.

Dimension hay còn gọi là bảng thứ nguyên là bảng có vai trò cung cấp ngữ cảnh và mô tả cho các dữ liệu trong bảng dữ kiện. Các bảng này đại diện cho các thực thể kinh doanh cần được lập mô hình, ví dụ như Sản phẩm, Nhân sự, Địa điểm, Kênh phân phối, Thời gian. Khác với bảng dữ kiện chứa các con số, bảng thứ nguyên chứa các giá

trị mang tính mô tả, văn bản để giúp xác định và phân loại dữ liệu. Về mặt kiến trúc, bảng thứ nguyên thường bao gồm một hoặc nhiều cấu trúc phân cấp để hỗ trợ việc phân loại dữ liệu từ tổng quan đến chi tiết. Trong trường hợp không có phân cấp, nó được gọi là thứ nguyên phẳng hoặc danh sách. Mỗi bảng thứ nguyên chứa các cột khóa chính hoạt động như mã định danh duy nhất, và các khóa này là một phần cấu thành nên khóa chính tổng hợp của bảng dữ kiện. Mặc dù đóng vai trò quan trọng trong việc định nghĩa ngữ cảnh, bảng thứ nguyên thường có số lượng hàng tương đối nhỏ và kích thước gọn nhẹ hơn nhiều so với bảng dữ kiện.



Hình 4.3. Kiến trúc mô hình dữ liệu Star Schema

4.4.1.2. Đặc điểm của Star Schema

Star Schema tạo ra một cơ sở dữ liệu chuẩn hóa ở mức logic, tối ưu cho truy vấn, nó tổ chức dữ liệu bằng cách xây dựng các bảng dimension bao quanh bảng fact giúp giảm các số lượng bảng cần join khi truy vấn.

Thiết kế linh hoạt, dễ dàng mở rộng trong suốt vòng đời của hệ thống, có thể thêm các dimension hoặc mở rộng thuộc tính trong dimension nếu nhu cầu tăng lên mà không gây xung đột trong hệ thống.

Phản ánh đúng cách người dùng cuối suy nghĩ và khai thác dữ liệu, có cấu trúc trực quan, gần với ngôn ngữ nghiệp vụ hơn là cấu trúc kỹ thuật.

Giảm độ phức tạp của siêu dữ liệu, có cấu trúc đơn giản, dễ mô tả, quản lý và bảo trì.

4.4.1.3. Ưu điểm và nhược điểm

Ưu điểm:

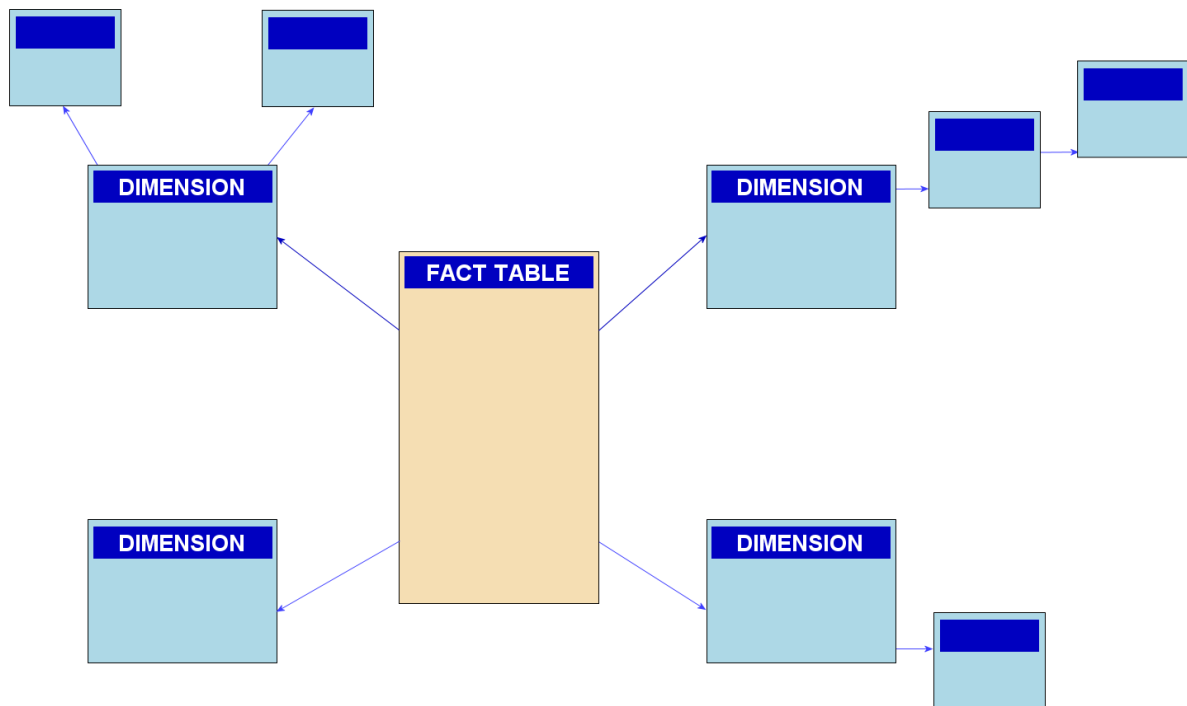
- Có hiệu suất truy vấn cao, mô hình dữ liệu có một số lượng hạn chế các bảng và các đường dẫn tham gia rõ ràng, cho nên tốc độ truy vấn nhanh hơn so với các hệ thống OLTP. Các truy vấn thường là trong các bảng thứ nguyên, nên hầu như thời gian truy vấn là tức thời, còn đối với các truy vấn lớn nối nhiều bảng thì chỉ cần vài giây hoặc vài phút để chạy.
- Nhờ sự đơn giản về mặt cấu trúc nên có tải hiệu suất và quản trị cao, vì thế có thể làm giảm thời gian cần thiết để tải hàng loạt bản ghi lớn vào cơ sở dữ liệu Star Schema.
- Có tính toàn vẹn tham chiếu được tích hợp sẵn, tính chất này được thực thi vì mỗi dữ liệu trong bảng dimension có một khóa chính duy nhất và các khóa chính này được tích hợp vào bảng fact làm khóa ngoại.
- Star Schema rất đơn giản và dễ hiểu, các bảng thứ nguyên chỉ được kết hợp với nhau thông qua bảng fact, các mối liên hệ này đại diện cho mối quan hệ giữa các bộ phận trong doanh nghiệp cho nên khách hàng cũng có thể duyệt qua các thuộc tính bảng thứ nguyên trước khi tạo truy vấn.

Nhược điểm:

- Mô hình Star Schema không mô tả tốt đối với các mối quan hệ nhiều-nhiều,
- Các bảng dimension có thể gặp khó khăn khi dữ liệu phức tạp, các dữ liệu trong dimension thường hay gộp lại vào một bảng nên đôi khi dữ liệu thường bị dư thừa, bảng chứa nhiều cột.
- Không phù hợp với các dữ liệu có cấu trúc phân cấp phức tạp, Star Schema chỉ làm việc hiệu quả đối với các dữ liệu đơn giản, ít phân cấp, khi dữ liệu quá phức tạp, có thể khiến cho mô hình khó mô tả rõ ràng cho dữ liệu.
- Chỉ phù hợp cho các báo cáo, phân tích hoặc thống kê, không phù hợp đối với hệ thống giao dịch, logic ràng buộc phức tạp, quan hệ chằng chéo giữa các thực thể.

4.4.2. Snowflake

Snowflake Schema được xem như là một mô hình dữ liệu nâng cấp từ mô hình Star Schema, khi dữ liệu trở nên phân cấp và quá phức tạp, mô hình Star Schema không thể biểu diễn tốt dữ liệu tốt trong một bảng dimension khi dữ liệu bị gộp, có thể dẫn đến dư thừa dữ liệu hoặc sai dữ liệu. Snowflake Schema phân tách, chuẩn hóa các bảng dimension thành các bảng dimension phụ trợ cho các bảng dimension trong Star Schema. Hình ảnh thiết kế Snowflake Schema khá giống với hình ảnh bông tuyết, có một bảng trung tâm ở giữa và nhiều nhánh các là bảng dimension liên kết với bảng trung tâm. Vì Snowflake Schema được nâng cấp từ Star Schema nên Snowflake mang đa số các đặc điểm tương tự như Star Schema.



Hình 4.4. Kiến trúc mô hình dữ liệu Snowflake Schema

4.4.2.1. Ưu điểm và nhược điểm của mô hình Snowflake

Ưu điểm:

- Có sự phát triển về hiệu suất truy vấn do các yêu cầu lưu trữ đĩa được giảm thiểu và tham gia các bảng tra cứu nhỏ hơn

- Nó cung cấp khả năng mở rộng lớn hơn trong mối quan hệ qua lại giữa các mức dimension và các thành phần.
- Không dư thừa nên dễ bảo trì hơn.

Nhược điểm:

- Do số lượng bảng tra cứu ngày càng tăng nên cần thêm nhiều sự nỗ lực bảo trì
- Có nhiều truy vấn phức tạp hơn nên có thể gây khó hiểu.
- Cần nhiều thời gian truy vấn hơn do hệ thống phải join dữ liệu từ nhiều bảng khác lại với nhau.

4.4.3. So sánh Star Schema và Snowflake

Tiêu chí	Star Schema	Snowflake Schema
Cách tổ chức đối tượng	Các đối tượng phụ xung quanh đối tượng chính được xây dựng chung trong một bảng dimension	Các đối tượng chính và đối tượng phụ được tách riêng thành nhiều bảng
Cấu trúc tổng thể	Các bảng dimension được xây dựng xung quanh một bảng fact	Bảng fact ở trung tâm, xung quanh là bảng đối tượng chính và các bảng đối tượng phụ liên kết tiếp
Số lượng JOIN	Chỉ cần một câu lệnh JOIN giữa bảng fact và các bảng dimension	Cần nhiều câu lệnh JOIN để truy xuất dữ liệu
Độ phức tạp thiết kế	Thiết kế cơ sở dữ liệu đơn giản	Thiết kế cơ sở dữ liệu phức tạp hơn
Cách lưu trữ dữ liệu	Dữ liệu được dàn trải theo chiều ngang, truy xuất nhanh	Dữ liệu được chuẩn hóa theo đúng đối tượng
Mức độ lặp dữ liệu	Dữ liệu bị lặp lại khá nhiều	Dữ liệu lặp ít hoặc không lặp

Tổ chức đối tượng chính	Một đối tượng chính được tổng hợp từ nhiều đối tượng phụ vào một bảng	Đối tượng chính được tách thành nhiều đối tượng phụ xung quanh
Hiệu năng truy vấn	Truy xuất dữ liệu theo nhu cầu (datacube) nhanh hơn	Truy xuất dữ liệu chậm hơn do phải JOIN nhiều bảng

Hình 4.5. Bảng so sánh Star Schema và Snowflake Schema [3]

4.5. Ưu điểm và nhược điểm của Data Warehouse

4.5.1. Ưu điểm

Giải quyết được bài dữ liệu rác hoặc dữ liệu phân mảnh bằng cách tập trung, thu thập dữ liệu từ nhiều nguồn, sau đó thực hiện chuẩn hóa, để tạo thành một bộ dữ liệu đồng nhất và dễ dàng sử dụng

Tạo ra được một nguồn dữ liệu duy nhất và chính xác cho toàn bộ doanh nghiệp

Hệ thống cung cấp thông tin chính xác và kịp thời để đưa ra các quyết định quan trọng trong quản lý quy trình kinh doanh, quản lý tài chính và tối ưu hàng hóa tồn kho.

Giúp doanh nghiệp dễ dàng nhận ra các cơ hội mới, cũng như nhìn thấy điểm mạnh, điểm yếu của thị trường để đi trước đối thủ.

4.5.2. Nhược điểm

Cần phải chi một lượng lớn chi phí cho cơ sở hạ tầng và chi phí duy trì cho lượng lớn dữ liệu, đặc biệt khi dữ liệu ngày càng mở rộng và phình to, khiến cho lượng chi phí này càng tăng.

Data Warehouse phụ thuộc vào nguồn dữ liệu gốc. Nếu dữ liệu đầu vào bị sai sót hoặc nhập liệu ẩu, nó sẽ ảnh hưởng trực tiếp đến tính toàn vẹn và độ tin cậy của thông tin được lưu trữ trong kho.

Việc gom dữ liệu từ quá nhiều nguồn khác nhau như khác định dạng hoặc khác cấu trúc là một bài toán kỹ thuật phức tạp. Nếu không xử lý tốt, nó có thể dẫn đến sự không nhất quán, mâu thuẫn dữ liệu trong hệ thống.

KẾT LUẬN

Tóm tắt nội dung

Ở mức tổng quát, các engine lưu trữ được chia thành hai loại chính: engine tối ưu hóa cho xử lý giao dịch (OLTP) và engine tối ưu hóa cho phân tích (OLAP). Có sự khác biệt lớn giữa các mô hình truy cập trong hai trường hợp sử dụng này:

- Hệ thống OLTP thường phục vụ trực tiếp người dùng, điều này có nghĩa là chúng có thể phải xử lý một khối lượng lớn các yêu cầu. Để chịu được tải này, các ứng dụng thường chỉ truy cập một số lượng nhỏ bản ghi trong mỗi truy vấn. Ứng dụng yêu cầu bản ghi thông qua một loại khóa nào đó, và engine lưu trữ sử dụng chỉ mục để tìm dữ liệu cho khóa được yêu cầu. Thời gian tìm kiếm của ổ đĩa thường là nút thắt cổ chai ở đây.
- Kho dữ liệu và các hệ thống phân tích tương tự ít được biết đến hơn, vì chúng chủ yếu được sử dụng bởi các nhà phân tích kinh doanh, không phải người dùng cuối. Chúng xử lý một khối lượng truy vấn thấp hơn nhiều so với các hệ thống OLTP, nhưng mỗi truy vấn thường đòi hỏi rất cao, cần quét hàng triệu bản ghi trong thời gian ngắn. Băng thông đĩa (không phải thời gian tìm kiếm) thường là nút thắt ở đây, và lưu trữ theo cột ngày càng trở thành giải pháp phổ biến cho loại công việc này.

Về phía OLTP, các công cụ lưu trữ từ hai trường phái chính:

- Trường phái cấu trúc nhật ký, chỉ cho phép ghi thêm vào các tệp và xóa các tệp lỗi thời, nhưng không bao giờ cập nhật một tệp đã được ghi. Bitcask, SSTables, LSM-trees, LevelDB, Cassandra, HBase, Lucene và các công cụ khác thuộc nhóm này.
- Trường phái cập nhật tại chỗ, coi đĩa như một tập hợp các trang có kích thước cố định có thể bị ghi đè. B-trees là ví dụ lớn nhất cho triết lý này, được sử dụng trong tất cả các cơ sở dữ liệu quan hệ chính và cả nhiều cơ sở dữ liệu phi quan hệ.

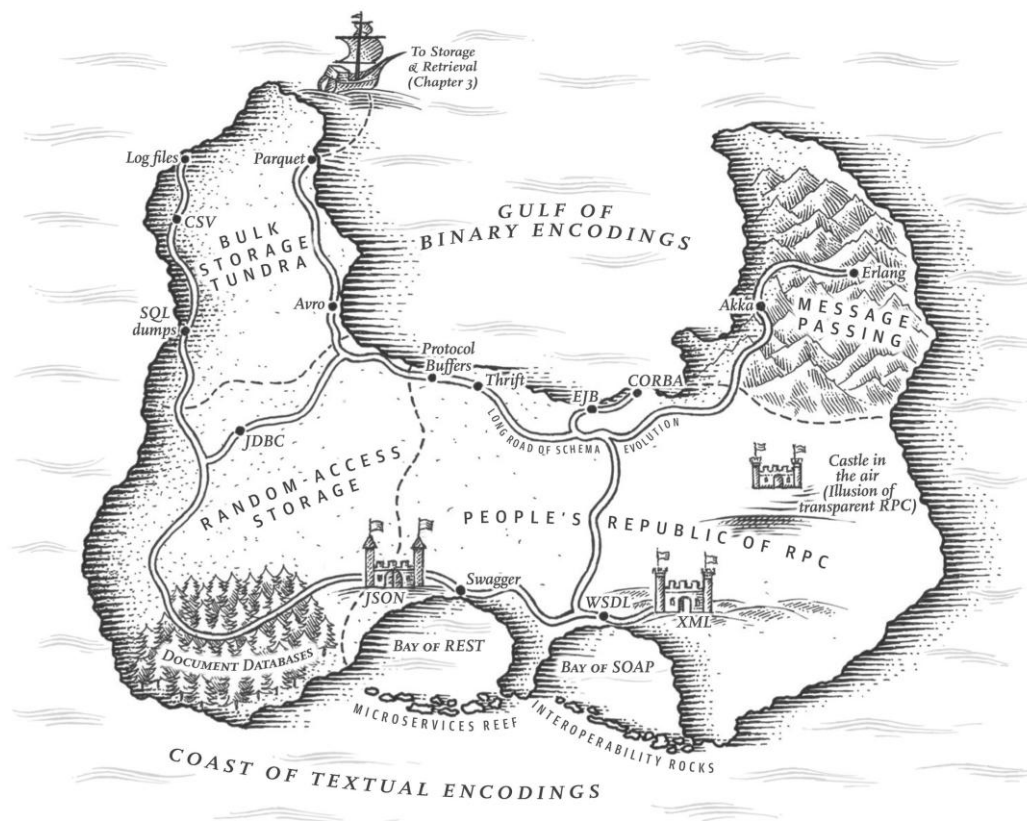
Các động cơ lưu trữ theo cấu trúc nhật ký là một phát triển tương đối mới. Ý tưởng chính của chúng là biến các ghi ngẫu nhiên thành các ghi tuần tự trên đĩa một

cách có hệ thống, điều này cho phép tăng thông lượng ghi nhờ vào đặc tính hiệu suất của ổ cứng và SSD.

Tiếp theo, khối lượng công việc phân tích lại khác biệt so với OLTP: khi các truy vấn yêu cầu quét tuần tự qua một số lượng lớn các hàng, các chỉ mục trở nên ít quan trọng hơn. Thay vào đó, việc mã hóa dữ liệu một cách thật gọn gàng trở nên quan trọng, nhằm giảm thiểu lượng dữ liệu mà truy vấn cần đọc từ đĩa. Cách lưu trữ theo cột đã giúp đạt được mục tiêu này.

Việc sở hữu kiến thức về hoạt động bên trong của các động cơ lưu trữ, sẽ giúp nhà phát triển ở vị thế tốt hơn nhiều khi biết công cụ nào phù hợp nhất cho ứng dụng cụ thể của mình. [4]

Explain map



TÀI LIỆU THAM KHẢO

- [1] Khoa Tran Dang, “LSM-Tree: Kẻ lười biếng đứng sau sức mạnh của NoSQL,” 8, 2025. [Trực tuyến]. Available: <https://www.linkedin.com/pulse/lsm-tree-k%E1%BA%BB-l%C6%B0%E1%BB%9Di-bi%E1%BA%BFng-%C4%91%E1%BB%A9ng-sau-s%E1%BB%A9c-m%E1%BA%A1nh-c%E1%BB%A7a-nosql-khoa-tran-dang-vtkwe/>. [Đã truy cập 15/01/2026].
- [2] [CTDL và GT] *Cây tìm kiếm tự cân bằng (B Tree) - Trực quan, dễ hiểu..* [Performance]. Học nhanh - Code chuẩn;, 2026.
- [3] Datapot.vn, “2 kiểu lưu trữ dữ liệu: Row-based storage vs Column-based storage,” 04/07/2024. [Trực tuyến]. Available: https://datapot.vn/luu-tru-du-lieu-row-based-storage-vs-column-based-storage/?srsltid=AfmBOooWMr3YHI4rjvZiHklxahLI-vo7L_IPGV53GIRj_X9BmX0RKb_h#Luu_tru_du_lieu_theo_Column-based_storage. [Đã truy cập 15/01/2026].
- [4] Martin Kleppmann, *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*, O'Reilly Media, 2017.
- [5] Viettel IDC, “Data Warehouse là gì? Tổng hợp kiến thức về kho dữ liệu,” 05/09/2024. [Trực tuyến]. Available: <https://viettelidc.com.vn/tin-tuc/data-warehouse-la-gi>. [Đã truy cập 15/1/2026].
- [6] Tenten.vn, “Data Warehouse là gì? Kiến thức cơ bản về kho dữ liệu,” 28/09/2023. [Trực tuyến]. Available: https://tenten.vn/tin-tuc/data-warehouse/#Ki7871n_truacutec_c7911a_Data_warehouse. [Đã truy cập 15/01/2026].
- [7] RENOVACLOUD, “Hiểu rõ Data Warehouse là gì chỉ trong 5 phút!,” [Trực tuyến]. Available: <https://renovacloud.com/data-warehouse-la-gi/>. [Đã truy cập 15/01/2026].

- [8] T. Phạm, “Phân biệt giữa Star schema và Snowflake schema,” 2023-02-05. [Trực tuyến]. Available: <https://vndataprodukt.org/phan-biet-giua-star-schema-va-snowflake-schema/>. [Đã truy cập 15/01/2026].